Verified Optimization

From Lattices to Solvers in Lean 4

A Zero-to-Hero Guide to Combinatorial Optimization
with a GW2 Build Editor Capstone

You built a propagator network for gear optimization.
It prunes beautifully. Now let's make it ***solve****.*

Companion to *Verified Order Theory* (the lattice/propagator book)

Contents

Preface

This book has an unusual origin.

It began with a practical problem: building a gear optimizer for Guild Wars 2's World vs. World game mode. You choose stat combinations for fourteen equipment slots, each drawn from a finite menu of options. You have stat targets to meet, a rune constraint that couples all six armor pieces, and an infusion budget for fine-tuning. The goal: find the loadout that gets as close to your targets as possible, penalising shortfalls much more than overshoots.

The first attempt was a *propagator network*—the architecture from our companion book *Verified Order Theory: From Partial Orders to Propagator Networks in Lean 4*. Each gear slot became a cell holding a set of remaining stat combos, ordered by reverse inclusion. Propagators enforced constraints: the rune propagator ensured all six armor rune slots matched; the stat-floor propagator detected when no remaining combination of gear could reach a target. Running the network to its Knaster–Tarski fixed point pruned the search space dramatically.

But propagation alone couldn't *solve* the problem. It could tell you that Dire gear on the headgear slot was impossible given your power target, but it couldn't tell you whether Marauder or Berserker on that slot led to a better overall build. Propagation eliminates the impossible; it doesn't choose the optimal among the possible.

This book picks up where propagation left off. It gives the propagator engine a driver: systematic search to explore the pruned space, bounding techniques to skip parts of the search that provably can't beat what you've already found, and heuristic methods to find good starting solutions fast. The result is a gear solver in Lean 4, developed alongside a *specification* of what it means for that solver to be correct—and, where the proofs are short enough, machine-checked verification of the pieces.

An honest word about "verified"

This is a book about *how to build and specify* a verified solver, not a finished verified artifact. Several of the deepest correctness theorems—solver soundness, branch-and-bound invariant preservation, LP-bound validity, infusion-DP optimality—are stated precisely as *specifications* and their proofs are left as extended exercises (shown in Lean with the `sorry` placeholder, which means "unproven"—see Appendix C). We are careful never to present a `sorry`-backed theorem as though it were machine-checked. What *is* genuinely proved (irreflexivity and transitivity of Pareto dominance, decidability of dominance, the small worked lemmas) is called out as such. Treat the capstone soundness theorem as a rigorously-stated contract with a proof sketch, in the spirit of a specification you would hand to a verification team—not as a closed proof.

This is a book about **discrete (combinatorial)** optimisation: the gear problem picks one combo per slot from a finite menu, so there are no gradients to descend. We survey the continuous world (convexity, linear programming) only where it feeds the discrete solver—LP relaxation as a bounding technique. Gradient descent and Newton's method are named for contrast and then set aside; they are not developed here. The mathematics we *do* develop is real: linear programming and duality, integer programming, branch and bound, constraint propagation,

Lagrangian relaxation, Pareto optimality, and dynamic programming—each motivated by the gear problem, implemented in Lean 4, and connected back to the lattice-theoretic foundations you already know.

Prerequisites and Lean setup

Comfort with reading Lean 4 (typeclasses, basic tactic proofs) and the material from *Verified Order Theory*—especially partial orders, lattices, join-semilattices, propagator cells, and the Knaster–Tarski fixed-point theorem. No deep proof-assistant experience is required: Appendix C, *A Lean 4 Primer*, explains every construct and tactic this book uses (`intro`, `exact`, `constructor`, `obtain`, `unfold`, `rfl`, `simp`, `omega`, `split`, and an honest account of `sorry`), each with a compiled example.

Toolchain and libraries. All Lean code targets toolchain `leanprover/lean4:v4.31.0`. The book mixes two kinds of listing. Most of the core data model, the heuristics, the infusion DP, and the Pareto proofs are **plain core Lean** (no external packages). But the propagators, the whole solver loop, and the A* skeleton depend on **Mathlib** (`Finset`, `SemilatticeSup`/`⊔`) and **Batteries** (`BinaryHeap`). Section 1.3 states this dependency explicitly and shows how to set up a Mathlib project. Every listing is machine-checked: the core ones against a bare toolchain (`OptimizationVerification.lean`), the Mathlib ones against Mathlib v4.31.0 (`OptimizationMathlibVerification.l`). Where a listing deliberately leaves a `sorry`, the surrounding text says so.

How to Read This Book

Part I opens with the build editor and your propagator attempt, then strengthens the propagator engine before introducing any new techniques. Part II develops the mathematical tools: linear programming, integer programming, search strategies. Part III adds heuristic methods and multi-objective optimization. Part IV assembles everything into the capstone solver with a formal correctness theorem.

Cross-reference boxes connect each new concept to its lattice-theoretic counterpart. You'll find that optimisation and order theory are deeply intertwined—more so than most textbooks suggest.

Let's build.

Part I

The Problem and the Engine You Already Have

Chapter 1

The Build Editor and the Propagator Attempt

> The question is not whether we can prune the search space,
> but whether we can navigate what remains.

1.1 The Gear Optimisation Problem

In Guild Wars 2's World vs. World mode, a character's combat effectiveness is determined by nine *attributes* (stats). Each stat has a different effect:

Stat	Effect
Power	Increases direct damage
Precision	Increases critical strike chance
Ferocity	Increases critical strike damage
Toughness	Increases armour
Vitality	Increases maximum health
Condition Damage	Increases damage-over-time
Expertise	Increases condition duration
Concentration	Increases boon duration
Healing Power	Increases outgoing healing

A character's stats come from three sources: base stats (determined by class), gear, and temporary buffs (food, utility, traits). This book concerns the gear component, which the player has full control over.

1.1.1 Equipment Slots

A full equipment loadout consists of:

Armour (6 pieces). Headgear, shoulders, coat, gloves, leggings, boots. Each piece has one rune slot and one infusion slot.

Weapons. The active weapon configuration is either:

- one two-handed weapon (2 sigil slots, 2 infusion slots), or
- one main-hand weapon + one off-hand weapon (1 sigil and 1 infusion each).

A character may have two weapon sets, but only the active set's stats count. We optimise only the active set.

Trinkets. Amulet (×1, 0 infusions), ring (×2, 3 infusions each), accessory (×2, 1 infusion each), back piece (×1, 2 infusions).

The total infusion budget is $6 + 2 + 6 + 2 + 2 = 18$ slots, each contributing +5 to a single stat of the player's choice. That's 90 freely distributable stat points.

1.1.2 Stat Combinations

Each gear piece is assigned a *stat combination* (also called a *prefix*). There are 40 WvW-legal stat combinations at Ascended rarity, falling into three categories:

Triple-attribute (23 combos). One *major* stat and two *minor* stats. Major stats receive a larger coefficient; minor stats receive a smaller one.

Quadruple-attribute (16 combos). Two major stats and two minor stats. Each major stat gets the 4-stat major coefficient; each minor stat gets the 4-stat minor coefficient.

Celestial (1 combo). All seven of Power, Precision, Ferocity, Vitality, Toughness, Healing Power, and Condition Damage receive the Celestial coefficient (equal for all). The WvW version of Celestial excludes Expertise and Concentration (nerfed relative to PvE).

The coefficients depend on the gear slot. At Ascended rarity and level 80:

Slot	3-stat Maj	3-stat Min	4-stat Maj	4-stat Min	Celestial
Armour					
Headgear	63	45	54	30	30
Shoulders	47	34	40	22	22
Coat	141	101	121	67	67
Gloves	47	34	40	22	22
Leggings	94	67	81	44	44
Boots	47	34	40	22	22
Weapons					
One-handed	125	90	108	59	59
Two-handed	251	179	215	118	118
Trinkets					
Amulet	157	108	133	71	72
Ring (×2)	126	85	106	56	57
Accessory (×2)	110	74	92	49	50
Back piece	63	40	52	27	28

For example, a Berserker's coat (3-stat: Power major, Precision and Ferocity minor) contributes 141 Power, 101 Precision, and 101 Ferocity. A Marauder coat (4-stat: Power and Precision major, Vitality and Ferocity minor) contributes 121 Power, 121 Precision, 67 Vitality, and 67 Ferocity.

1.1.3 Formalising in Lean 4

Lean 4 --- Core Data Model

```
inductive StatType where
  | power | precision | ferocity | toughness | vitality
  | conditionDamage | expertise | concentration | healingPower
  deriving DecidableEq, Repr, Hashable

inductive GearSlot where
  | headgear | shoulders | coat | gloves | leggings | boots
  | mainhand | offhand    -- or combined as twoHand
  | amulet | ring1 | ring2 | accessory1 | accessory2 | backPiece
  deriving DecidableEq, Repr

inductive ComboCategory where
  | triple   -- 1 major, 2 minor
  | quad     -- 2 major, 2 minor
  | celestial -- 7 equal
  deriving DecidableEq, Repr

structure StatCombo where
  name     : String
  category : ComboCategory
  majors   : List StatType
  minors   : List StatType
  deriving DecidableEq, Repr
```

The stat contribution of a combo on a given slot is determined by which category the combo belongs to and whether a given stat is major, minor, or (for Celestial) one of the seven included stats:

Lean 4 --- Stat Coefficients

```
structure SlotCoefficients where
  tripleMajor  : Nat
  tripleMinor  : Nat
  quadMajor    : Nat
  quadMinor    : Nat
  celestial    : Nat
  deriving Repr

def coefficients : GearSlot → SlotCoefficients
  | .headgear   => ⟨63,  45,  54,  30, 30⟩
  | .shoulders  => ⟨47,  34,  40,  22, 22⟩
  | .coat       => ⟨141, 101, 121, 67, 67⟩
  | .gloves     => ⟨47,  34,  40,  22, 22⟩
  | .leggings   => ⟨94,  67,  81,  44, 44⟩
  | .boots      => ⟨47,  34,  40,  22, 22⟩
  | .mainhand   => ⟨125, 90,  108, 59, 59⟩  -- 1H
  | .offhand    => ⟨125, 90,  108, 59, 59⟩  -- 1H
```

```
  | .amulet     => (157, 108, 133, 71, 72)
  | .ring1      => (126, 85,  106, 56, 57)
  | .ring2      => (126, 85,  106, 56, 57)
  | .accessory1 => (110, 74,  92,  49, 50)
  | .accessory2 => (110, 74,  92,  49, 50)
  | .backPiece  => (63,  40,  52,  27, 28)

/-- Contribution of `combo` on `slot` to `stat`. -/
def contribution (slot : GearSlot) (combo : StatCombo)
    (stat : StatType) : Nat :=
  let c := coefficients slot
  match combo.category with
  | .triple =>
    if combo.majors.contains stat then c.tripleMajor
    else if combo.minors.contains stat then c.tripleMinor
    else 0
  | .quad =>
    if combo.majors.contains stat then c.quadMajor
    else if combo.minors.contains stat then c.quadMinor
    else 0
  | .celestial =>
    let wvwStats : List StatType :=
      [.power, .precision, .ferocity, .toughness,
       .vitality, .conditionDamage, .healingPower]
    if wvwStats.contains stat then c.celestial else 0
```

Before defining a build, we need three enumerations—the list of all gear slots, the list of all stat types, and a menu of stat combos—plus a placeholder `Rune` type. These are referenced throughout the book, so we give them once here (a fuller combo menu appears in Appendix A):

Lean 4 --- Enumerations and Runes

```
def GearSlot.all : List GearSlot :=
  [.headgear, .shoulders, .coat, .gloves, .leggings, .boots,
   .mainhand, .offhand,
   .amulet, .ring1, .ring2, .accessory1, .accessory2, .backPiece]

def StatType.all : List StatType :=
  [.power, .precision, .ferocity, .toughness, .vitality,
   .conditionDamage, .expertise, .concentration, .healingPower]

/-- A rune (placeholder: runes couple the six armour slots; the rune
    propagator lives in the lattice book). -/
structure Rune where
  name : String
  deriving DecidableEq, Repr

/-- A small illustrative combo menu (the full 40 are in Appendix A). -/
def allCombos : List StatCombo :=
  [("Berserker's", .triple, [.power], [.precision, .ferocity]),
```

```
  ⟨"Knight's",    .triple, [.toughness], [.power, .precision]⟩,
  ⟨"Marauder",    .quad, [.power, .precision], [.vitality, .ferocity]⟩,
  ⟨"Minstrel's",  .quad, [.toughness, .healingPower],
                         [.vitality, .concentration]⟩,
  ⟨"Celestial",   .celestial, [], []⟩]
```

A *build* assigns a stat combo to each slot, a rune, and a distribution of infusions:

Lean 4 --- Build and Stat Total

```
structure Build where
  gear      : GearSlot → StatCombo
  rune      : Rune
  infusions : StatType → Nat
  -- Invariant: ∑ infusions = 18

/-- Total stat from gear + infusions (excluding base stats and buffs). -/
def statTotal (b : Build) (s : StatType) : Nat :=
  let gearTotal := (GearSlot.all.map fun slot =>
    contribution slot (b.gear slot) s).sum
  gearTotal + 5 * b.infusions s
```

There are 14 gear slots, so `GearSlot.all` has 14 entries; the search-space and termination bounds later in the book count from this list.

1.1.4 The Objective

Given stat targets $T : \texttt{StatType} \to \mathbb{N}$, we define the penalty:

Asymmetric Penalty

$$\text{pen}(b) \;=\; \sum_{s \in \texttt{StatType}} w_{\text{miss}} \cdot \max\big(0,\; T(s) - \text{statTotal}(b,s)\big) \;+\; w_{\text{over}} \cdot \max\big(0,\; \text{statTotal}(b,s) - T(s)\big)$$

where $w_{\text{miss}} \gg w_{\text{over}}$ (e.g. $10 : 1$). Missing a target by 100 costs ten times as much as exceeding it by 100.

Lean 4 --- Penalty

```
def penalty (target actual : Nat) (wMiss wOver : Nat) : Nat :=
  if actual < target then
    wMiss * (target - actual)
  else
    wOver * (actual - target)

def totalPenalty (b : Build) (targets : StatType → Nat)
    (wMiss wOver : Nat) : Nat :=
  (StatType.all.map fun s =>
    penalty (targets s) (statTotal b s) wMiss wOver).sum
```

```
-- Fixed penalty weights used throughout the book's listings.
def wMiss : Nat := 10
def wOver : Nat := 1
```

The optimisation problem: find the build b^* that minimises $\text{pen}(b^*)$ subject to the rune constraint (all six armour runes identical), the weapon constraint (2H or 1H+1H, not a mix), and the infusion budget ($\sum_s \texttt{infusions}(s) = 18$).

Soft objective vs. hard floors: a modelling decision

The penalty above is a *soft* objective: missing a target is penalised (heavily), not forbidden. But the propagators and solver we build from Chapter 3 onward treat each stat target as a **hard floor**: the `LinearGeq` propagator *deletes* any combo that cannot reach a target. These are two *different* problems, and it matters which one we are solving:

- **Soft-penalty problem:** minimise $\text{pen}(b)$ over *all* builds. Here a small shortfall on one stat may be the best outcome.
- **Hard-floor problem (what the solver decides):** among builds that meet *every* target ($\text{statTotal}(b, s) \geq T(s)$ for all s), minimise the (overshoot) penalty; report infeasible if none exist.

The solver in Part IV solves the *hard-floor* problem, and its soundness theorem (Chapter 12) is stated against a feasibility predicate that includes the floors. This is a legitimate and common choice (targets as requirements), but it is **not** the same as minimising the soft penalty: `LinearGeq` pruning can discard the soft-penalty optimum (a combo giving the *smallest* shortfall—hence lowest penalty—may still fail to reach the floor and be pruned). We flag this again where the propagator is defined, and the correctness theorem is stated precisely to match the code.

1.1.5 The Search Space

With 40 stat combos and the 14 gear slots of our `GearSlot` type, the raw search space is $40^{14} \approx 6.9 \times 10^{22}$ gear assignments—before considering rune choices, sigils, and infusion distributions. Brute force is out of the question.

1.2 The Propagator Attempt

Having read *Verified Order Theory*, you recognised the structure: each gear slot is a finite-domain variable, the rune constraint couples six of them, and the stat targets impose global arithmetic constraints. This is a *constraint satisfaction problem*, and you built a propagator network to attack it.

1.2.1 The Domain-Reduction Lattice

Cross-Reference: Lattice Book

In the lattice book, we defined a *cell* as a value in a join-semilattice, updated by joining new information. The set-of-possible-values lattice ordered by reverse inclusion appeared as Example (d) in the chapter on propagator network design. We use that exact lattice here.

For each gear slot i, define the *domain cell* D_i as a subset of the 40 WvW-legal stat combos,

ordered by reverse inclusion:

$$D_i \subseteq D_j \iff D_j \subseteq D_i \qquad \text{(fewer remaining options = more information).}$$

The initial value is the full set of 40 combos ($\bot$ = "nothing ruled out"). The join (merge) is intersection: $D_i \sqcup D_j = D_i \cap D_j$ (keep only combos consistent with both sources of information). The top element $\top$ is the empty set: contradiction (all combos ruled out).

The product lattice $\prod_i \text{Finset}(\texttt{StatCombo})^{\text{op}}$ represents the full network state.

1.2.2 Propagators You Built

The rune propagator. If any rune is eliminated from any armour slot's rune options, eliminate it from all six. If any rune is fixed (the domain becomes a singleton), fix all six. This is a global `AllEqual` constraint.

The stat-floor propagator. For each stat type s and each unfilled slot i, compute

$$\text{maxContrib}(i, s) = \max\{\texttt{contribution}(i, c, s) \mid c \in D_i\}.$$

If $\sum_i \text{maxContrib}(i, s) + 90 < T(s)$ (even with all infusions devoted to s), the problem is infeasible: signal $\top$.

Running to fixpoint. Fire all propagators repeatedly until no domain changes. By the Knaster–Tarski theorem (proven in the lattice book), this converges to the least fixed point of the propagator system—the maximal amount of pruning achievable by this set of propagators.

1.3 Which Parts Need Mathlib

The next listing is the first in the book to reach outside core Lean. A `Cell` is parametrised by a `SemilatticeSup` (a join-semilattice), and a gear-slot domain is a `Finset StatCombo`—both from **Mathlib**, the Lean community's mathematics library. Later, the A* skeleton of Chapter 6 uses `BinaryHeap` from **Batteries**. Core Lean knows none of these. So the book's listings split into two groups:

- **Core Lean** (no packages): the data model, the penalty, the heuristics (hill climbing, simulated annealing, tabu, GA), the infusion DP, the Pareto dominance proofs, and the `BitVec` performance code. These are machine-checked in `OptimizationVerification.lean`.
- **Mathlib/Batteries**: everything that mentions `Finset`, `SemilatticeSup`/$\sqcup$, or `BinaryHeap`—the propagators, the backtracking/branch-and-bound solver, and A*. These are machine-checked in `OptimizationMathlibVerification.lean`.

Setting up a Mathlib project

Create a project directory with a `lean-toolchain` file containing the single line `leanprover/lean4:v4.31.0`, and a `lakefile.toml`:

```
name = "gearsolver"
defaultTargets = ["GearSolver"]

[[require]]
name = "mathlib"
scope = "leanprover-community"
rev = "v4.31.0"
```

```
[[lean_lib]]
name = "GearSolver"
```

Then run `lake update` to resolve the dependency and `lake exe cache get` to download prebuilt `.olean` binaries—do not skip this, or your machine will spend hours compiling Mathlib from source. All fetched packages and build artifacts live under the project's `.lake/` directory. Begin the Mathlib-dependent files with `import Mathlib` (Batteries comes with it; the A* code additionally does `open Batteries`). The core-Lean listings need no project at all—`lake env lean File.lean` against the bare toolchain suffices.

Lean 4 (Mathlib) --- Propagator Network (from the Lattice Book)

```
-- Reusing infrastructure from Verified Order Theory
structure Cell (L : Type) [SemilatticeSup L] where
  value : L

def Cell.update [SemilatticeSup L] (c : Cell L) (new : L) : Cell L :=
  ⟨c.value ⊔ new⟩

-- Domain cell: Finset StatCombo with reverse-inclusion order
-- ⊔ is ∩ (intersection), ⊥ is allCombos, ⊤ is ∅

-- The propagator loop from the lattice book:
partial def runToFixpoint (network : PropNetwork) : PropNetwork :=
  let network' := network.fireAll
  if network' == network then network'
  else runToFixpoint network'
```

1.3.1 Where It Gets Stuck

The propagator network does its job: domains shrink, impossible combos are eliminated, and infeasible problems are detected early. But after convergence, you're left with *pruned domains*—perhaps 8–15 combos remaining per slot instead of 40. That's still $15^{14} \approx 2.9 \times 10^{16}$ possible builds.

Three gaps remain:

1. **Propagation doesn't search.** It tells you what's *impossible*, not what's *best*. After fixpoint, multiple combos remain per slot, and propagation has no mechanism for choosing among them.
2. **Propagation doesn't optimise.** It can detect infeasibility (a domain goes empty, signalling $\top$) but has no notion of "this build scores lower penalty than that one." There is no *objective function* in the propagator framework.
3. **Propagation doesn't handle tradeoffs.** Should you sacrifice 20 Power to gain 50 Vitality? That depends on the penalty weights and how close you are to each target. Propagation doesn't model comparative reasoning.

Intuition

Think of propagation as a *filter*: it removes options that provably can't be part of any good solution. A *solver* is a filter plus a *navigator*: it explores the remaining options systematically, tracking the best solution found so far, and proving (by exhaustion or

bounding) that no unexplored option can beat it.

The rest of this book fills these three gaps. Chapter 2 maps the optimisation landscape. Chapter 3 makes propagation itself stronger (so the filter removes more). Parts II and III add the navigator. Part IV assembles everything into a verified solver.

Chapter 2

The Landscape of Optimisation

> All happy optimisation problems are alike; each unhappy one is unhappy in its own way.

Now that we have a concrete problem and understand what propagation gives us (and doesn't), let's place the build editor in the broader landscape of optimisation.

2.1 What Is an Optimisation Problem?

Optimisation Problem

An *optimisation problem* consists of:

- A set of *decision variables* $x_1, \ldots, x_n$.
- For each variable x_i, a *domain* D_i of possible values.
- A set of *constraints* C that restrict which combinations of values are allowed.
- An *objective function* $f : D_1 \times \cdots \times D_n \to \mathbb{R}$ to minimise (or maximise).

A *feasible solution* is an assignment $x_i \in D_i$ satisfying all constraints. An *optimal solution* is a feasible solution minimising f.

Lean 4 --- OptProblem

```
structure OptProblem (V : Type) (D : V → Type) where
  constraints : (∀ v, D v) → Prop
  objective   : (∀ v, D v) → Int

structure Solution (P : OptProblem V D) where
  assignment : ∀ v, D v
  feasible   : P.constraints assignment

def isOptimal (P : OptProblem V D) (sol : Solution P) : Prop :=
  ∀ other : Solution P, P.objective sol.assignment ≤ P.objective other.assignment
```

The build editor is an instance: variables are gear slots, domains are stat combos, constraints are the rune and weapon and infusion rules, and the objective is the asymmetric penalty. Concretely, a *gear problem* bundles the stat targets, the per-slot initial domains, and (for the scalarisation sweep of Chapter 14) an optional per-stat miss-weight:

Lean 4 (Mathlib) --- GearProblem

```
structure GearProblem where
  targets        : StatType → Nat
  initialDomains : GearSlot → Finset StatCombo
  -- optional per-stat miss weight (defaults to the global wMiss = 10);
  -- used only by the Pareto scalarisation sweep of Chapter 14
  wMissPerStat   : StatType → Nat := fun _ => wMiss
```

2.2 Continuous vs. Discrete

In *continuous* optimisation, domains are subsets of $\mathbb{R}^n$. Gradient descent, Newton's method, and convex optimisation live here.

In *discrete* (or *combinatorial*) optimisation, domains are finite sets. The build editor is discrete: you can't choose "60% Berserker + 40% Marauder"—you pick one combo per slot. This means:

- **No gradients.** The objective is not differentiable (it's defined on a finite set). Gradient descent doesn't apply.
- **Local optima everywhere.** Unlike convex problems, where every local minimum is global, discrete problems can have many local optima separated by barriers.
- **NP-hardness lurks.** Many discrete optimisation problems are NP-hard. We don't expect polynomial-time exact algorithms.

2.3 Convexity: Why It's Not Enough

Convex Set

A set $S \subseteq \mathbb{R}^n$ is *convex* if for all $x, y \in S$ and $\lambda \in [0,1]$: $\lambda x + (1-\lambda)y \in S$.

Convex Function

A function $f : S \to \mathbb{R}$ on a convex set S is *convex* if for all $x, y \in S$ and $\lambda \in [0,1]$:

$$f\big(\lambda x + (1-\lambda)y\big) \;\leq\; \lambda f(x) + (1-\lambda) f(y).$$

Convex ⇒ Local = Global

If f is convex and x^* is a local minimum of f, then x^* is a *global* minimum.

Proof. Suppose x^* is a local minimum but not global: there exists y with $f(y) < f(x^*)$. By convexity, for small $\lambda > 0$:

$$f\big((1-\lambda)x^* + \lambda y\big) \leq (1-\lambda) f(x^*) + \lambda f(y) < f(x^*).$$

But $(1-\lambda)x^* + \lambda y$ is arbitrarily close to x^*, contradicting the local minimality of x^*. □

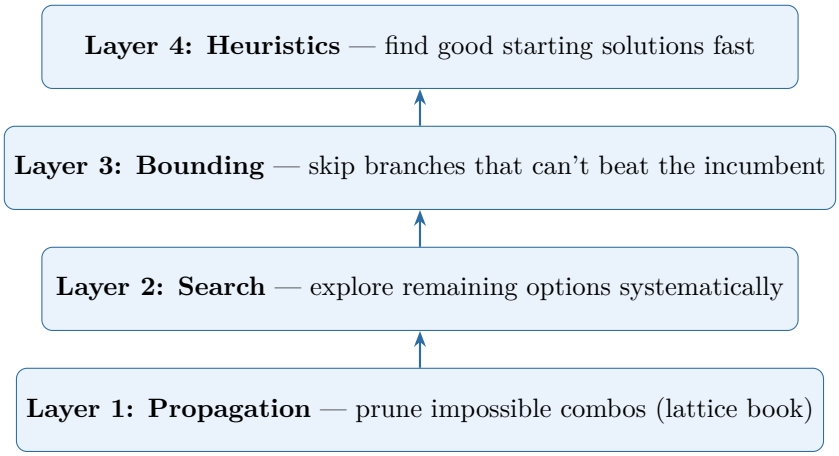
This is why gradient descent works for convex problems: you can't get trapped in a local minimum. But the build editor's domain is a finite set of points, not a convex region of $\mathbb{R}^n$. The concept of "moving a small distance toward a better solution" doesn't apply.

Intuition

We'll exploit convexity indirectly: by *relaxing* the discrete problem to a continuous one (pretending we can choose fractional amounts of each stat combo), solving the relaxation, and using the solution as a *bound* on how good any discrete solution can be. This is the LP relaxation idea of Chapter 4.

2.4 The Solver Architecture

Here is the plan. The gear optimiser is a stack of four layers:



Each layer wraps around the ones below it. Propagation (Layer 1) is the *foundation*—it runs at every node of the search tree. Search (Layer 2) explores the pruned space. Bounding (Layer 3) uses LP relaxation to prune entire subtrees. Heuristics (Layer 4) provide a warm start so bounding can prune more aggressively.

Cross-Reference: Lattice Book

Layer 1 is precisely the propagator network from *Verified Order Theory*. The contribution of this book is Layers 2–4. But notice: propagation isn't just the foundation—it runs *inside* every search node, making the entire solver dramatically faster.

Chapter 3

Making Propagation Stronger

> Before adding new machinery, squeeze more out of what you have.

Before we introduce search and bounding, let's strengthen the propagator engine itself. The propagators from the lattice book (rune equality, basic stat-floor) are correct but leave information on the table. This chapter adds three new propagators that prune significantly more.

3.1 The LinearGeq Propagator

The basic stat-floor propagator detects infeasibility when the total maximum possible contribution falls below a target. The `LinearGeq` propagator goes further: it prunes individual combos from individual slots.

LinearGeq Propagator

For each stat type s and each slot i, and for each combo $c \in D_i$: compute the maximum possible total stat from all *other* slots plus c's contribution from slot i plus the full infusion budget:

$$U(i, c, s) = \texttt{contribution}(i, c, s) + \sum_{j \neq i} \text{maxContrib}(j, s) + 90.$$

If $U(i, c, s) < T(s)$, then no build using combo c on slot i can meet target $T(s)$—even if every other slot takes the best possible combo for s and all infusions go to s. Remove c from D_i.

Lean 4 (Mathlib) --- LinearGeq Propagator

```
/-- Max contribution to `stat` achievable by any combo left in `domain`
    on `slot` (0 if the domain is empty). -/
def maxContribForStat (domain : Finset StatCombo) (slot : GearSlot)
    (stat : StatType) : Nat :=
  domain.fold max 0 (fun c => contribution slot c stat)

/-- Remove combos from slot i that can't contribute enough to stat s. -/
def linearGeqPropagate
    (domains : GearSlot → Finset StatCombo)
    (targets : StatType → Nat)
```

```
    (slot : GearSlot) : Finset StatCombo :=
  domains slot |>.filter fun combo =>
    StatType.all.all fun stat =>
      let myContrib := contribution slot combo stat
      let othersMax := (GearSlot.all.filter (· ≠ slot) |>.map fun j =>
        maxContribForStat (domains j) j stat).sum
      let infusionBudget := 90  -- 18 slots × 5 each
      myContrib + othersMax + infusionBudget ≥ targets stat
```

LinearGeq is sound for hard floors, not for the soft penalty

`LinearGeq` removes c from D_i exactly when combo c cannot reach $T(s)$ even granting slot i the whole 90-point infusion budget and every other slot its best combo. This is a valid *necessary* condition for the **hard-floor** problem: any build meeting all floors keeps every combo it uses (this is the honest restatement `propagation_preserves_feasible` of Chapter 12). It is *not* sound for the pure soft-penalty objective: the very combo that misses a floor by the smallest amount may be the soft-penalty minimiser, yet `LinearGeq` deletes it. Throughout, the solver commits to the hard-floor reading.

Monotonicity of LinearGeq

The `LinearGeq` propagator is monotone: if domains shrink (more information), the propagator can only remove *more* combos (produce more information), never fewer.

Proof. Suppose $D'_j \subseteq D_j$ for all j (domains have shrunk in the reverse-inclusion order, meaning we have more information). For any slot i and combo c:

$$\sum_{j \neq i} \text{maxContrib}'(j, s) \;\leq\; \sum_{j \neq i} \text{maxContrib}(j, s)$$

because maxContrib can only decrease when the domain shrinks. So $U'(i, c, s) \leq U(i, c, s)$. Any combo removed by the old propagator is still removed by the new one (with smaller U), and possibly additional combos are removed. Hence the output domain D'_i under the new inputs is a subset of the output under the old inputs—i.e., the propagator is monotone in the reverse-inclusion order. □

3.2 The LinearLeq Propagator

The dual of `LinearGeq`: if we want to minimise overshoot, we can prune combos that would cause unavoidable excess.

LinearLeq Propagator

For each stat s, slot i, and combo $c \in D_i$: compute

$$L(i, c, s) = \texttt{contribution}(i, c, s) + \sum_{j \neq i} \text{minContrib}(j, s).$$

If $L(i, c, s) > T(s) + \text{maxAllowedOvershoot}$, remove c from D_i. This is weaker (since the overshoot penalty weight is low), but still helps when a stat is close to its target.

Lean 4 (Mathlib) --- LinearLeq Propagator

```
/-- Min contribution to `stat` over the combos left in `domain`
    (0 when the domain is empty; see the caveat below). -/
def minContribForStat (domain : Finset StatCombo) (slot : GearSlot)
    (stat : StatType) : Nat :=
  (domain.image (fun c => contribution slot c stat)).min.getD 0

/-- Remove combos from slot i that would force unavoidable overshoot
    on some stat beyond the allowed threshold. -/
def linearLeqPropagate
    (domains : GearSlot → Finset StatCombo)
    (targets : StatType → Nat)
    (maxOvershoot : Nat)  -- e.g., 200 stat points
    (slot : GearSlot) : Finset StatCombo :=
  domains slot |>.filter fun combo =>
    StatType.all.all fun stat =>
      let myContrib := contribution slot combo stat
      let othersMin := (GearSlot.all.filter (· ≠ slot) |>.map fun j =>
        minContribForStat (domains j) j stat).sum
      myContrib + othersMin ≤ targets stat + maxOvershoot
```

Monotonicity of LinearLeq (nonempty domains)

`LinearLeq` is monotone *as long as no domain is empty*: if the domains shrink but stay nonempty, the propagator removes at least as many combos.

Proof. When a *nonempty* domain shrinks ($\emptyset \neq D'_j \subseteq D_j$), the minimum contribution from other slots can only *increase* (the minimum over a smaller nonempty set is $\geq$ the minimum over a larger one): $\text{minContrib}'(j, s) \geq \text{minContrib}(j, s)$. So $L'(i, c, s) \geq L(i, c, s)$, and any combo removed by the old propagator is still removed. □

Warning

The minimum of an *empty* set has no natural value. We use the convention $\text{minContrib}(\emptyset, s) = 0$ (via `.getD 0` above), which makes `minContribForStat` total, but it breaks the "min over a smaller set is larger" step: an empty domain reports 0, the *smallest* possible value. Since an empty domain already signals infeasibility (handled separately by the solver), this is harmless in practice; but the monotonicity claim genuinely needs the nonempty hypothesis stated above. This is a real gap the earlier edition glossed over.

Note that `LinearLeq` is most useful for stats where the player explicitly *doesn't want* too much. In WvW, this might occur when a player wants to maximise damage stats while keeping Toughness below a threshold (to avoid drawing aggro from certain AI enemies that target the highest-Toughness player).

3.3 The Infusion Budget Propagator

Infusion Budget Propagator

After all gear propagation has reached fixpoint, compute, for each stat s:

$$\text{bestGear}(s) = \sum_i \text{maxContrib}(i, s).$$

The *deficit* for s is $\max(0, T(s) - \text{bestGear}(s))$. If $\sum_s \text{deficit}(s) > 90$ (the total deficit across all stats exceeds the infusion budget), the problem is infeasible.

This is a global constraint that reasons about the interaction between gear choices and the infusion budget. It can detect infeasibility that no single-stat propagator would catch: each stat's deficit might be small enough to fill, but the total across all stats exceeds the budget.

Lean 4 (Mathlib) --- Infusion Budget Propagator

```
/-- Check whether the total stat deficit across all stats exceeds
    the infusion budget (18 slots × 5 = 90 points). -/
def infusionBudgetFeasible
    (domains : GearSlot → Finset StatCombo)
    (targets : StatType → Nat) : Bool :=
  let totalDeficit := StatType.all.map (fun stat =>
    let bestGear := (GearSlot.all.map fun slot =>
      maxContribForStat (domains slot) slot stat).sum
    if targets stat > bestGear then targets stat - bestGear
    else 0) |>.sum
  totalDeficit ≤ 90
```

3.4 The Strengthened Network

Combining all propagators into a single network:

1. Fire the rune propagator (from the lattice book).
2. Fire `LinearGeq` for every slot and stat.
3. Fire `LinearLeq` for every slot and stat (if overshoot control is enabled).
4. Check the infusion budget propagator.
5. Repeat until fixpoint.

Termination of the Strengthened Network

The strengthened propagator network terminates in at most $\sum_i |D_i^{(0)}|$ steps, where $D_i^{(0)}$ is the initial domain of slot i. With 40 combos and 14 slots, this is at most 560 steps.

Proof. Each propagation step either removes at least one combo from at least one domain (strictly decreasing the lattice height) or is a no-op (fixpoint reached). The lattice height is $\sum_i |D_i|$, initially at most $40 \times 14 = 560$. Since the height is a natural number that strictly decreases at each non-trivial step, the process terminates in at most 560 steps. □

Exercise

Implement the full strengthened propagator network in Lean 4, reusing the `Cell` and `Propagator` types from the lattice book. Run it on the following problem: Heavy armour class, stat targets Power ≥ 2500, Precision ≥ 2000, Ferocity ≥ 1200. How many combos remain per slot after propagation?

Exercise

Prove in Lean 4 that the `LinearGeq` propagator satisfies the monotonicity property from the lattice book: if input domains shrink, output domains shrink.

Part II

The Mathematics of Optimisation

Chapter 4

Linear Programming

The shortest distance between two points is a straight line,
and the best solution to a linear problem is at a corner.

4.1 The LP Standard Form

Linear Program (Standard Form)

$$\min_{x \in \mathbb{R}^n} \; c^\top x \quad \text{subject to} \quad Ax \geq b, \quad x \geq 0.$$

Here $c \in \mathbb{R}^n$ is the cost vector, $A \in \mathbb{R}^{m \times n}$ is the constraint matrix, and $b \in \mathbb{R}^m$ is the right-hand side. We use the *greater-than* form $Ax \geq b$ because the gear problem's stat targets are *floors* (statTotal $\geq T$); it is equivalent to the $\leq$ form by negating rows.

Transpose and inner-product notation

For column vectors $c, x \in \mathbb{R}^n$, the *transpose* $c^\top$ is the row vector $(c_1, \ldots, c_n)$, so $c^\top x = \sum_{i=1}^{n} c_i x_i$ is the ordinary *dot (inner) product*—a single number, the weighted total cost. For a matrix $A \in \mathbb{R}^{m \times n}$, the transpose $A^\top \in \mathbb{R}^{n \times m}$ swaps rows and columns, and the identity $(A^\top y)^\top x = y^\top (Ax)$ (both equal $\sum_{i,j} A_{ij}\, x_j\, y_i$) is the one algebraic fact the duality proof below leans on. If you have not met this notation, read $c^\top x$ as "dot product of c and x" and $A^\top y$ as "A with rows and columns flipped, times y".

The feasible region $\{x \mid Ax \geq b, x \geq 0\}$ is a convex *polyhedron*—an intersection of finitely many half-spaces.

Fundamental Theorem of Linear Programming

If an LP has an optimal solution, then it has an optimal solution at a *vertex* (extreme point) of the feasible polytope.

Intuition

Why? The objective $c^\top x$ is a linear function. On any edge of the polytope, it is monotonically increasing or decreasing. So moving along an edge toward a vertex can only improve (or maintain) the objective. The optimum must occur where you can't improve further—at a vertex.

4.2 The Simplex Method

The simplex method exploits this: start at a vertex, move to an adjacent vertex with a better objective value, repeat until no improving neighbour exists.

Cross-Reference: Lattice Book

The vertices of the feasible polytope, ordered by objective value, form a partial order. The simplex method is a *monotone walk* on this structure—the same pattern as propagator convergence. In the lattice book, cell values only moved upward; here, the objective only moves downward (since we're minimising). The fixed point is the optimum.

4.2.1 Pivoting

A vertex of the polytope corresponds to a *basic feasible solution* (BFS): a choice of m constraints to hold with equality (the *basis*). The simplex method maintains a basis and performs *pivot* operations to swap one constraint in and one out, moving to an adjacent vertex.

The pivot rule selects which non-basic variable to bring into the basis (improving the objective) and which basic variable to remove (maintaining feasibility). Bland's rule (choose the smallest-index improving variable) guarantees termination by preventing cycling.

Lean 4 --- Simplex Sketch

```
structure Tableau where
  m : Nat  -- number of constraints
  n : Nat  -- number of variables
  tab : Array (Array Float)  -- (m+1) × (n+m+1) augmented matrix
  basis : Array Nat          -- indices of basic variables

/-- One pivot step: bring variable enterIdx into the basis. -/
def pivot (T : Tableau) (enterIdx : Nat) : Option Tableau :=
  -- Minimum ratio test to find the leaving variable
  let leaveIdx? := minRatioTest T enterIdx
  match leaveIdx? with
  | none => none  -- unbounded
  | some leaveIdx =>
    -- Gaussian elimination to pivot
    some (gaussianPivot T enterIdx leaveIdx)

/-- Run simplex to completion. -/
partial def simplex (T : Tableau) : SimplexResult :=
  match blandEnteringVariable T with
  | none => .optimal (extractSolution T)  -- no improving variable
  | some enterIdx =>
```

```
    match pivot T enterIdx with
    | none => .unbounded
    | some T' => simplex T'
```

4.3 Duality

Every LP has a *dual* problem. For the primal min $c^\top x$ s.t. $Ax \geq b, x \geq 0$, the dual is max $b^\top y$ s.t. $A^\top y \leq c, y \geq 0$. (One constraint direction flips relative to the other because the dual lower-bounds a minimisation.)

Weak Duality

For any primal feasible x and dual feasible y: $c^\top x \geq b^\top y$.

Proof. $c^\top x \geq (A^\top y)^\top x = y^\top Ax \geq y^\top b = b^\top y$. The first inequality uses dual feasibility ($A^\top y \leq c$ and $x \geq 0$, so $(A^\top y)^\top x \leq c^\top x$); the second uses primal feasibility ($Ax \geq b$ and $y \geq 0$, so $y^\top Ax \geq y^\top b$). Thus every dual feasible y gives a *lower* bound $b^\top y$ on the primal minimum $c^\top x$—exactly what a bounding technique needs. □

Strong Duality

If the primal has an optimal solution x^*, then the dual has an optimal solution y^* with $c^\top x^* = b^\top y^*$.

The dual provides *certificates of optimality*: if you exhibit primal and dual solutions with matching objective values, you've *proved* the primal solution is optimal, without trusting the simplex algorithm's internal state.

Intuition

In the gear solver, the LP relaxation gives a *lower bound* on the achievable penalty. If we also produce the dual solution, we have a *machine-checkable certificate* that no gear loadout can do better than this bound. When the integer solution matches the LP bound, optimality is proven.

4.4 LP Relaxation of the Gear Problem

Here is how to relax the build editor to an LP. For each slot i and combo c, introduce a variable $x_{i,c} \in [0, 1]$ representing the "fraction" of combo c used on slot i. The constraints:

1. $\sum_c x_{i,c} = 1$ for each slot i (exactly one combo per slot).
2. $x_{i,c} \geq 0$ for all i, c.
3. The stat contributions are linear in x: $\text{statTotal}(x, s) = \sum_{i,c} x_{i,c} \cdot \texttt{contribution}(i, c, s)$.
4. Linearise the penalty: introduce auxiliary variables for the max-with-zero terms.

The LP relaxation drops the constraint $x_{i,c} \in \{0, 1\}$ and allows fractional values. The optimal LP value is a lower bound on the optimal integer value.

Exercise

Formulate the LP relaxation for a small instance (3 armour slots, 5 stat combos, 2 stat targets). Solve it by hand or with the simplex implementation. Verify that the LP

optimum is $\leq$ the best integer solution.

Chapter 5

Integer Programming and Branch-and-Bound

Divide and conquer, but know when to surrender.

5.1 The Integrality Gap

The LP relaxation allows fractional solutions like "use 60% Berserker and 40% Marauder on the coat." The *integrality gap* is the difference between the LP optimum and the best integer (actual gear) solution.

Integrality Gap

$$\text{gap} = \text{opt(IP)} - \text{opt(LP)}$$

where opt(IP) is the optimal integer (actual gear) penalty and opt(LP) is the optimal LP relaxation penalty. Since $\text{opt(LP)} \leq \text{opt(IP)}$ (relaxation can only improve the objective), the gap is always ≥ 0.

Intuition

Consider a simplified two-slot, two-stat problem. Slot A can be Berserker's (+141 Power, +0 Toughness) or Knight's (+0 Power, +141 Toughness). Slot B is the same. Targets: Power ≥ 100, Toughness ≥ 100.
LP relaxation: use 50% Berserker + 50% Knight on each slot. Each slot contributes 70.5 Power and 70.5 Toughness. Total: 141 Power, 141 Toughness. Both targets met. Penalty = 0.
Best integer solution: one slot Berserker's, one slot Knight's. Total: 141 Power, 141 Toughness. Penalty = 0.
Here the gap is zero—the LP and IP optima coincide. But now add a third stat target (Ferocity ≥ 50) where Berserker's gives 101 Ferocity and Knight's gives 0. The LP can get Ferocity from a fractional Berserker assignment on both slots; the IP must concentrate it on one, potentially overshooting other targets. The gap opens up.

The gap can be large: the LP might achieve penalty 50 by artfully spreading stat contributions across fractional combos, while the best integer build has penalty 200 because it must commit each slot to a single combo.

Historical Note

The study of integrality gaps is central to the theory of approximation algorithms. Nemhauser and Wolsey's *Integer and Combinatorial Optimization* (1988) established much of the foundational theory. A small gap means the LP relaxation gives a tight bound and B&B converges quickly; a large gap means the LP bound is loose and more search is needed. For the gear problem, the gap is typically small when targets are tight (close to the maximum achievable) and larger when targets are loose.

5.2 Branch and Bound

Branch and bound (B&B) is a systematic search that exploits the LP bound to prune.

Branch and Bound

1. **Initialise.** Set `incumbent` (best solution found) to ∞. Create a root node representing the full problem.
2. **Select.** Pick an unexplored node from the queue.
3. **Bound.** Solve the LP relaxation of this node's subproblem. If the LP optimum $\geq$ `incumbent`, *prune* (no solution in this subtree can beat the incumbent).
4. **Branch.** If not pruned and the LP solution is fractional, choose a slot i with a fractional assignment. Create child nodes, one for each remaining combo in D_i (fixing slot i to that combo).
5. **Propagate.** At each child node, run the propagator network. If any domain becomes empty, prune.
6. **Repeat.** Go to step 2 until the queue is empty. The incumbent is optimal.

Cross-Reference: Lattice Book

The B&B search tree is a *lattice of subproblems*, ordered by refinement (fixing more variables). At each node, the LP bound is an upper bound on the subtree's optimal value. Pruning = detecting that a subtree is "dead"—analogous to detecting $\top$ (contradiction) in a propagator cell. The whole B&B is a monotone process: as we descend the tree, domains only shrink, bounds only tighten.

5.2.1 The B&B Invariant

The key correctness invariant, which we'll formalise and prove in the capstone:

B&B Invariant

At every point during the B&B search:

1. The incumbent is a *feasible* solution (satisfies all constraints).
2. Every pruned node n satisfies LP(n) $\geq$ `incumbent.penalty` (its LP bound is no better than the incumbent).
3. Every integer solution in a pruned subtree has penalty $\geq$ LP(n) $\geq$ `incumbent.penalty` (by weak duality and the relaxation relationship).

When the queue is empty, these invariants imply the incumbent is optimal: every feasible solution is either the incumbent itself, or lives in a pruned subtree with penalty $\geq$ incumbent.

5.2.2 Variable and Value Ordering

The order in which we branch on slots and try combos dramatically affects the number of nodes explored.

Fail-first variable ordering. At each branch node, choose the slot with the *smallest remaining domain* (after propagation). Intuition: if a slot has only 2 remaining combos, branching on it creates 2 children. If another has 15, branching creates 15 children. Fail-first focuses effort where the problem is most constrained.

Lean 4 (Mathlib) --- Fail-First Variable Selection

```
/-- Select the unfixed slot with the smallest remaining domain. -/
def selectSlotFailFirst
    (domains : GearSlot → Finset StatCombo) : GearSlot :=
  let unfixed := GearSlot.all.filter fun s => (domains s).card > 1
  match unfixed with
  | []            => .headgear   -- no unfixed slot: caller is at a leaf
  | first :: rest =>
    rest.foldl (init := first) fun best slot =>
      if (domains slot).card < (domains best).card then slot else best
```

Note the explicit `match`: the earlier edition wrote `unfixed.head!`, which *panics at runtime* when every domain is already a singleton (`unfixed` empty). Pattern-matching makes the "all fixed" case total and lets the caller detect a leaf.

Best-first value ordering. Within a slot, try combos in order of their contribution to the *tightest* stat target first. Define "tightest" as the stat with the largest remaining deficit (target minus current best achievable):

Lean 4 (Mathlib) --- Value Ordering by Tightest Deficit

```
/-- Find the stat with the largest remaining deficit. -/
def tightestStat (domains : GearSlot → Finset StatCombo)
    (targets : StatType → Nat) : StatType :=
  StatType.all.foldl (init := .power) fun best stat =>
    let bestGear := (GearSlot.all.map fun s =>
      maxContribForStat (domains s) s stat).sum
    let deficit := if targets stat > bestGear then targets stat - bestGear else 0
    let bestDeficit := if targets best > (GearSlot.all.map fun s =>
      maxContribForStat (domains s) s best).sum then targets best -
      ↪ (GearSlot.all.map fun s =>
      maxContribForStat (domains s) s best).sum else 0
    if deficit > bestDeficit then stat else best

/-- Order combos by contribution to the tightest stat (descending).
    NOTE: domains are enumerated as LISTS here. `Finset.toList` is
    noncomputable in Mathlib, so a runnable solver either keeps
    list-backed domains (as below) or uses `Finset.sort`. -/
def orderCombos (slot : GearSlot) (combos : List StatCombo)
    (tightest : StatType) : List StatCombo :=
```

```
  combos.mergeSort fun a b =>
    contribution slot a tightest > contribution slot b tightest
```

This increases the chance of finding a good solution early in the search, improving the incumbent and enabling more LP-based pruning.

Finsets are for reasoning; lists are for running

Mathlib's `Finset.toList` is `noncomputable` (it picks a representative via choice), so any solver that enumerates a `Finset` domain with `.toList` *cannot execute*. We keep `Finset` in the propagator listings because it is the right object for the monotonicity proofs, but the executable solver represents each domain as a *sorted list* of surviving combos (equivalently, enumerate a `Finset` with `Finset.sort`). The one fully runnable end-to-end component in this book—the infusion DP of Chapter 10—is written in core Lean over lists and arrays, and we `#eval` it on a concrete instance there.

5.2.3 Symmetry Breaking

Ring and Accessory Symmetry

Ring 1 and Ring 2 are interchangeable: swapping their stat combos produces an identical build. The same holds for Accessory 1 and Accessory 2.

Impose a total order on stat combos (e.g. alphabetical by name) and require $\texttt{combo}(\texttt{ring}_1) \leq \texttt{combo}(\texttt{ring}_2)$. This halves the search space for each symmetric pair without eliminating any essentially different builds.

Lean 4 --- Symmetry Breaking (specification)

```
/-- Any total order on combos will do; here, by name length then name. -/
def comboOrd (c : StatCombo) : Nat := c.name.length

/-- A build is canonical if symmetric slot pairs are ordered. -/
def Build.isCanonical (b : Build) : Prop :=
  comboOrd (b.gear .ring1) ≤ comboOrd (b.gear .ring2) ∧
  comboOrd (b.gear .accessory1) ≤ comboOrd (b.gear .accessory2)

-- SPECIFICATION (proof left as an extended exercise): every build has a
-- canonical equivalent with the same penalty. `sorry` marks it UNPROVEN.
theorem canonical_exists (b : Build) :
    ∃ b', b'.isCanonical ∧ totalPenalty b' = totalPenalty b := by
  -- Swap rings/accessories if needed; stat totals are unchanged because
  -- contribution depends only on the slot type, not the index
  -- (coefficients .ring1 = coefficients .ring2).
  sorry
```

This theorem is stated as a *specification*: the `sorry` means it is **unproven** in this book (see Appendix C). The proof is a good exercise—the key fact is `coefficients .ring1 = coefficients .ring2`, so swapping the two rings leaves every `statTotal` and hence the penalty unchanged.

Exercise

Prove the `canonical_exists` theorem. Hint: you need the fact that `contribution .ring1 = contribution .ring2` (the coefficient table gives the same values for both ring slots).

Chapter 6

Search Strategies

> Search is the art of not looking where you don't need to.

6.1 Backtracking Search with Propagation

The simplest complete search strategy: choose a variable (slot), try each value (combo), propagate, recurse.

Lean 4 (Mathlib) --- Backtracking Search

```
-- `enumerate` lists a domain's combos (Finset.toList is noncomputable,
-- so a runnable solver filters a reference list by membership, or keeps
-- list-backed domains); `propagateToFixpoint`, `extractBuild`, `fixSlot`
-- are the strengthened-network helpers from Chapter 3.

/-- Backtracking search with propagation at each node. -/
partial def backtrack
    (domains : GearSlot → Finset StatCombo)
    (assigned : List (GearSlot × StatCombo))
    (best : Option (Build × Nat))  -- incumbent
    (targets : StatType → Nat) : Option (Build × Nat) :=
  -- 1. Propagate
  let domains' := propagateToFixpoint domains
  -- 2. Check for empty domain (contradiction)
  if GearSlot.all.any (fun s => (domains' s).card = 0) then best
  -- 3. Check if all assigned
  else if GearSlot.all.all (fun s => (domains' s).card = 1) then
    let build := extractBuild domains'
    let pen := totalPenalty build targets wMiss wOver
    match best with
    | none => some (build, pen)
    | some (_, bestPen) =>
      if pen < bestPen then some (build, pen) else best
  -- 4. Branch on smallest domain (fail-first)
  else
```

```
    let slot := selectSlotFailFirst domains'
    (enumerate (domains' slot)).foldl (init := best) fun acc combo =>
      let domains'' := fixSlot domains' slot combo
      backtrack domains'' ((slot, combo) :: assigned) acc targets
```

Two fixes over the earlier edition: Mathlib's `Finset` has no `isEmpty` field (test `card = 0`), and `Finset.fold` folds a *commutative–associative* operation, not a general accumulator loop—so branching enumerates the domain to a list and uses `List.foldl`.

6.1.1 Worked Example: A Tiny Gear Problem

To build intuition, consider a stripped-down problem with three armour slots (coat, leggings, boots), three stat combos (Berserker's, Marauder, Celestial), and a single stat target: Power ≥ 300.

The Power contributions at Ascended:

	Berserker's (3-stat Maj)	**Marauder** (4-stat Maj)	**Celestial** (Celestial)
Coat	141	121	67
Leggings	94	81	44
Boots	47	40	22

First, a sanity check that decides everything below. The largest Power the three slots can produce *from gear alone* is $141 + 94 + 47 = 282 < 300$. So the target is unreachable without infusions, and reachable only because the propagator credits each combo with the full 90-point infusion budget. Let us run the propagator exactly as defined (recall $U(i, c, \text{Power}) = \texttt{contribution}(i, c, \text{Power}) + \sum_{j \neq i} \text{maxContrib}(j, \text{Power}) + 90$; keep c iff $U \geq 300$). These numbers are machine-checked (`ch6Survivors` in `OptimizationVerification.lean`).

Step 1a: No infusions (+0). If we drop the +90 term, every combo on every slot fails. The best any slot can do is its own max plus the other two slots' maxes: coat sees $141 + 94 + 47 = 282 < 300$, and every other combo/slot is smaller. So `LinearGeq` empties *every* domain, and the instance is reported **infeasible**. (The earlier edition's claim that propagation leaves {Berserker's, Marauder} on each slot is wrong—nothing survives.)

Step 1b: With the +90 infusion budget (as the propagator is actually defined). Now compute U per slot:

- **Coat** (maxContrib of others $= 94+47 = 141$): Berserker's $141+141+90 = 372$; Marauder $121 + 141 + 90 = 352$; Celestial $67 + 141 + 90 = 298 < 300$. *Remove Celestial from coat*; keep Berserker's and Marauder.
- **Leggings** (others $= 141 + 47 = 188$): Berserker's $94 + 188 + 90 = 372$; Marauder $81 + 188 + 90 = 359$; Celestial $44 + 188 + 90 = 322 \geq 300$. *All three survive.*
- **Boots** (others $= 141+94 = 235$): Berserker's $47+235+90 = 372$; Marauder $40+235+90 = 365$; Celestial $22 + 235 + 90 = 347 \geq 300$. *All three survive.*

So LinearGeq removes exactly one combo—Celestial on the coat—and the search space drops from $3^3 = 27$ to $2 \cdot 3 \cdot 3 = 18$.

Step 2: Search. Propagation alone did not decide the problem, so we branch. The coat has the smallest domain (2), so fail-first picks it. Fixing the coat and evaluating leaves the solver to distribute the 90 infusion points to clear the residual Power deficit and minimise overshoot

on the other stats—which is exactly the infusion subproblem of Chapter 10. For instance, all-Berserker's gear gives 282 Power, needing $\lceil (300 - 282)/5 \rceil = 4$ infusions on Power to reach the floor, leaving 14 for the other targets.

Step 3: Result. The instance is feasible *only* once the 90-point infusion budget is credited: gear tops out at 282, and $282 + 90 = 372 \geq 300$. The solver returns the floor-meeting build of minimum penalty. The lesson: the +90 term in `LinearGeq` is not optional bookkeeping—remove it and this very instance flips from feasible to infeasible.

6.2 A* and Best-First Search

Instead of depth-first search (which explores deeply before backtracking), best-first search maintains a priority queue of nodes ordered by their LP bound. It always expands the most promising node first.

A* for Optimisation

Maintain a priority queue of search nodes, ordered by $f(n) = g(n) + h(n)$ where:
- $g(n)$ = cost of the partial assignment so far (penalty from already-fixed slots).
- $h(n)$ = heuristic lower bound on the remaining cost (LP relaxation of unfixed slots).

Always expand the node with smallest $f(n)$.

Admissibility of LP Heuristic

If $h(n) \leq h^*(n)$ (the LP bound is a lower bound on the true remaining cost), then A* finds an optimal solution.

Proof. Suppose A* terminates by expanding a goal node n^* (a complete assignment). At that moment, every unexplored node m in the queue satisfies $f(m) \geq f(n^*)$ (A* always expands the smallest-f node). For any unexplored node m: $f(m) = g(m) + h(m) \leq g(m) + h^*(m)$ by admissibility. But $g(m)+h^*(m)$ is the best objective achievable through m. So the best objective through any unexplored path is $\geq f(n^*) = g(n^*)$ (since $h(n^*) = 0$ for a complete assignment). Therefore n^* is optimal. □

Lean 4 (Batteries) --- A* Search Skeleton

```
open Batteries  -- BinaryHeap lives in Batteries

structure SearchNode where
  domains  : GearSlot → Finset StatCombo
  assigned : List (GearSlot × StatCombo)
  gCost    : Nat   -- penalty from fixed slots
  hCost    : Nat   -- LP bound on remaining
  -- (no `deriving Repr`: `domains` is a function, so Repr cannot be derived)

def SearchNode.fCost (n : SearchNode) : Nat := n.gCost + n.hCost

/-- A* search for the optimal gear build.  We want the SMALLEST fCost first,
    and `BinaryHeap.extractMax` pops the largest element, so the comparator
    must be REVERSED (`>`): the "max" under `>` is the min fCost. -/
partial def aStarSearch
```

```
    (queue : BinaryHeap SearchNode (·.fCost > ·.fCost))
    (incumbent : Option (Build × Nat))
    (targets : StatType → Nat) : Option (Build × Nat) :=
  -- extractMax returns `(Option node, heap)`, NOT `Option (node, heap)`
  match queue.extractMax with
  | (none, _) => incumbent  -- queue empty: incumbent is optimal
  | (some node, queue') =>
    -- Pruning: if f(node) ≥ incumbent, skip
    match incumbent with
    | some (_, bestPen) =>
      if node.fCost ≥ bestPen then aStarSearch queue' incumbent targets
      else expandNode node queue' incumbent targets
    | none => expandNode node queue' incumbent targets
```

This is a standard result from AI search theory. The LP relaxation satisfies admissibility because relaxing integrality constraints can only improve (lower) the objective. Three subtleties the earlier edition got wrong: `SearchNode` cannot `deriving Repr` (its `domains` field is a function); Batteries' `BinaryHeap` offers only `extractMax`, returning a *pair* (an `Option` node and the rest of the heap) rather than an `Option` of a pair; and because `extractMax` pops the *largest* element, the comparator must be *reversed* (`·.fCost > ·.fCost`) so that "largest under $>$" is the smallest f-cost that best-first search actually wants.

Historical Note

A* was invented by Peter Hart, Nils Nilsson, and Bertram Raphael in 1968 at the Stanford Research Institute. The "A" stands for "Algorithm" (it was called Algorithm A with admissible heuristics, hence A*). It remains the gold standard for optimal search with heuristics. In our solver, A* with LP-relaxation heuristics is the combination of Layers 2 and 3 (search + bounding) from the architecture diagram.

6.3 Nogood Learning

When backtracking from a dead end, we can record the combination of assignments that caused the failure and prevent future exploration of any branch that contains the same combination.

Nogood

A *nogood* is a partial assignment that is provably inconsistent. If $\{\text{coat} \mapsto \text{Celestial}, \text{leggings} \mapsto \text{Celestial}\}$ causes propagation to detect infeasibility (Power target unachievable), then this pair is a nogood. Any future branch containing both assignments can be pruned immediately.

Lean 4 --- Nogood Store

```
/-- A nogood is a set of (slot, combo) assignments known to be infeasible. -/
structure Nogood where
  assignments : List (GearSlot × StatCombo)

/-- Check if the current assignment contains any known nogood. -/
def isNogoodHit (nogoods : List Nogood) (assigned : List (GearSlot × StatCombo))
```

```
    : Bool :=
  nogoods.any fun ng =>
    ng.assignments.all fun (slot, combo) =>
      assigned.any fun (s, c) => s == slot && c == combo

/-- Extract a nogood from a failed propagation.  The whole failing partial
    assignment is a SOUND (if not minimal) nogood: that exact combination
    drove propagation to the empty domain. -/
def extractNogood (assigned : List (GearSlot × StatCombo))
    (failedSlot : GearSlot) (failedStat : StatType)
    : Nogood :=
  ⟨assigned⟩
```

Minimising a nogood soundly is subtle

The earlier edition kept only assignments with `contribution slot combo failedStat > 0`. That is **unsound** for a stat-*floor* (LinearGeq) failure: the culprits of a floor miss are exactly the slots contributing *too little*—often those contributing *zero* to the failed stat—so dropping them can yield a set that is *not* inconsistent, wrongly pruning feasible branches. The safe extractor above keeps the entire failing assignment. Shrinking a nogood soundly (retaining only a minimal inconsistent core) requires re-checking that the retained subset still forces the floor miss—a QuickXplain-style minimisation, left as an exercise.

Cross-Reference: Lattice Book

This is the same pattern as CDCL (conflict-driven clause learning) in SAT solving, which we discussed in the lattice book as "adding a new propagator at runtime." A nogood is a constraint learned from a conflict; adding it to the network prevents the same conflict from recurring. In the lattice-theoretic view, a nogood is a new propagator that removes assignments from the product lattice of domain cells.

Exercise

Extend the backtracking search from earlier in this chapter to maintain a nogood store. When propagation fails, extract a nogood and add it. Before exploring each branch, check the nogood store. Measure the reduction in explored nodes on a test instance.

Exercise

Prove in Lean 4 that nogood learning preserves completeness: if a build was optimal before learning, it is still reachable after learning. Hint: nogoods only prune infeasible partial assignments.

Chapter 7

Relaxation and Bounding

To understand the integer world,
first solve the fractional one.

7.1 Lagrangian Relaxation

Instead of relaxing integrality, we can relax *constraints*. The rune constraint (all six armour rune slots must match) is a coupling constraint that prevents us from optimising each slot independently.

Lagrangian Relaxation

Move the rune constraint into the objective as a penalty:

$$L(\lambda) = \min_{x} \ \mathrm{pen}(x) + \sum_{i<j} \lambda_{ij} \cdot \mathbb{1}[\mathrm{rune}(i) \neq \mathrm{rune}(j)]$$

where $\lambda_{ij} \geq 0$ are *Lagrange multipliers*. For any $\lambda \geq 0$, $L(\lambda)$ is a lower bound on the optimal penalty.

Weak Lagrangian Duality

For any $\lambda \geq 0$: $L(\lambda) \leq \mathrm{opt}(\text{original problem})$.

Proof. Let x^* be an optimal solution to the original problem. Since x^* satisfies the rune constraint, $\mathbb{1}[\mathrm{rune}(i) \neq \mathrm{rune}(j)] = 0$ for all i, j in x^*. Therefore:

$$L(\lambda) \leq \mathrm{pen}(x^*) + \sum_{i<j} \lambda_{ij} \cdot 0 = \mathrm{pen}(x^*) = \mathrm{opt}\,.$$

The inequality holds because $L(\lambda)$ is a minimum over *all* assignments (including ones violating the rune constraint), while x^* is just one specific feasible assignment. □

With the rune constraint relaxed, the problem decomposes into independent per-slot subproblems, each trivially solvable by enumeration (pick the best combo for each slot independently).

7.1.1 Subgradient Optimisation

The *Lagrangian dual* is $\max_{\lambda \geq 0} L(\lambda)$: find the multipliers that give the tightest bound. This is a non-smooth optimisation problem (the function L is piecewise linear), solved by the subgradient method:

Lean 4 --- Subgradient Method for Lagrangian Dual

```
/-- All unordered pairs from a list (used for the C(6,2) rune couplings). -/
def List.pairs : List α → List (α × α)
  | []      => []
  | x :: xs => (xs.map fun y => (x, y)) ++ pairs xs

/-- One iteration of subgradient optimisation. -/
def subgradientStep (lambda : Array Float) (stepSize : Float)
    (solution : GearSlot → StatCombo) : Array Float :=
  -- Subgradient: violation of the rune constraint
  -- g_ij = 1 if rune(i) ≠ rune(j), else 0
  let armourSlots : List GearSlot :=
    [.headgear, .shoulders, .coat, .gloves, .leggings, .boots]
  let violations := (List.pairs armourSlots).map fun (i, j) =>
    if runeOf (solution i) == runeOf (solution j) then 0.0 else 1.0
  -- Update: λ' = max(0, λ + stepSize * g).  In v4.31 `Array.zipWith`
  -- takes the FUNCTION first, and it is `max`, not `Float.max`.
  Array.zipWith (fun li gi => max 0.0 (li + stepSize * gi))
    lambda violations.toArray

/-- Run the subgradient method for a fixed number of iterations. -/
def lagrangianDual (problem : GearProblem) (iters : Nat := 100)
    : Float × (GearSlot → StatCombo) :=
  let init_lambda := Array.replicate 15 0.0  -- C(6,2) = 15 pairs
  let result := (List.range iters).foldl
    (init := ((0.0 : Float), init_lambda,
              (none : Option (GearSlot → StatCombo))))
    fun (bestBound, lambda, _bestSol) k =>
      -- Solve Lagrangian subproblem (per-slot enumeration)
      let solution := solveRelaxedPerSlot lambda problem.targets
      let bound := lagrangianObjective solution lambda problem.targets
      -- DIMINISHING step: depends on the CURRENT iteration index k,
      -- so t_k → 0 as k grows (a genuine diminishing schedule).
      let stepSize := 1.0 / (1.0 + k.toFloat)
      let lambda' := subgradientStep lambda stepSize solution
      let bestBound' := max bestBound bound
      (bestBound', lambda', some solution)
  match result with
  | (bound, _, sol) => (bound, sol.getD defaultSolution)
```

Five corrections over the earlier edition: there is no `List.pairs` in the library (we define it); in v4.31 `Array.zipWith` takes the *function first*; `Float.max` is spelled `max`; `Array.mkArray` was renamed `Array.replicate`; and the fold's 3-tuple result is unpacked with a `match` (the old `|>.map` used `Prod.map`, which needs two functions). Crucially, the step size now uses the *current* index `k`, not the fixed total `iters`—the earlier `1.0 / (1.0 + iters.toFloat)` was **constant** across all

iterations, contradicting the "diminishing" comment and prose.

Historical Note

Lagrangian relaxation was pioneered by Marshall Fisher and Michael Held in the 1970s for vehicle routing and scheduling problems. The key insight is that *complicating constraints*—the ones that prevent decomposition—can be moved into the objective, trading hard constraints for soft penalties. The resulting decomposition often reveals beautiful structure: in our case, the rune constraint is the only thing preventing each slot from being optimised independently.

7.2 Reduced-Cost Fixing

Reduced-Cost Fixing

After solving the LP relaxation, each non-basic variable $x_{i,c}$ has a *reduced cost* $\bar{c}_{i,c}$: the amount the objective would increase if we forced $x_{i,c}$ to become positive. If LP $+\bar{c}_{i,c} \geq$ `incumbent`, then any solution using combo c on slot i has penalty $\geq$ incumbent. We can permanently remove c from D_i.

This is *propagation informed by relaxation*: the LP relaxation feeds information back into the propagator network, tightening domains beyond what pure constraint reasoning can achieve.

Lean 4 (Mathlib) --- Reduced-Cost Fixing

```
/-- Remove combos whose reduced cost proves they can't be in
    any solution better than the incumbent. -/
def reducedCostFix
    (domains : GearSlot → Finset StatCombo)
    (lpResult : LPResult)
    (incumbentPen : Nat) : GearSlot → Finset StatCombo :=
  fun slot => (domains slot).filter fun combo =>
    let rc := lpResult.reducedCost slot combo
    -- Keep combo only if LP bound + reduced cost < incumbent
    lpResult.objectiveValue + rc < incumbentPen.toFloat
```

Cross-Reference: Lattice Book

Reduced-cost fixing bridges the LP world and the propagator world. The LP relaxation is "above" the propagator network in the solver stack, but its conclusions ("combo c is too expensive for slot i") are fed back *downward* as domain reductions—exactly the kind of information propagator cells consume. In the lattice book's terms, reduced-cost fixing is an additional propagator whose input is the LP solution and whose output is a tightened domain.

7.2.1 Comparing Bound Quality

Different bounding techniques give different quality bounds:

Method	Bound quality	Computation cost
Propagation only	Weakest	Cheapest (milliseconds)
Lagrangian relaxation	Medium	Moderate (seconds)
LP relaxation	Tightest	Most expensive (per node)

In practice, we use propagation at every B&B node (cheap), LP relaxation at the root and at nodes where propagation doesn't prune enough (moderate cost), and Lagrangian relaxation as an alternative when the LP is too expensive.

Exercise

Implement the Lagrangian relaxation for the gear problem. Run the subgradient method for 100 iterations with a diminishing step size. Compare the Lagrangian bound to the LP relaxation bound on the same problem instance. How close are they?

Exercise

Prove in Lean 4 that if the Lagrangian dual bound equals the incumbent's penalty, then the incumbent is optimal. Hint: use the weak Lagrangian duality theorem.

Part III

Heuristics and Multi-Objective Optimisation

Chapter 8

Local Search and Metaheuristics

> Perfect is the enemy of good—but good is the friend of perfect.

Everything up to now has been exact: propagation is sound, LP bounds are valid, B&B is complete. This chapter introduces *heuristic* methods that find good solutions fast, without guaranteeing optimality. Their role in the solver: provide a strong incumbent to warm-start B&B, so bounding can prune more aggressively from the start.

8.1 Neighbourhood Structure

Neighbourhood for Gear Builds

Two builds are *neighbours* if they differ in exactly one slot's stat combo. The *neighbourhood* $N(b)$ of build b is the set of all builds reachable by changing one slot. $|N(b)| \leq 14 \times 39 = 546$.

The neighbourhood defines the "moves" available to local search. Its size is a tradeoff: a larger neighbourhood (e.g., changing two slots at once, giving $|N| \approx 14^2 \times 39^2 \approx 300{,}000$) explores more options per step but takes longer per step. The single-slot neighbourhood is the standard choice for gear optimisation because it's large enough to escape many local optima but small enough that evaluating all neighbours is fast.

Lean 4 --- Neighbourhood Enumeration

```
/-- Generate all single-slot neighbours of a build. -/
def allNeighbours (b : Build) : List Build :=
  GearSlot.all.flatMap fun slot =>   -- `List.bind` was removed in v4.31
    allCombos.filter (· ≠ b.gear slot) |>.map fun combo =>
      { b with gear := fun s => if s == slot then combo else b.gear s }

/-- A neighbour move: which slot changed and to what. -/
structure Move where
  slot  : GearSlot
  combo : StatCombo

/-- Generate all moves with their resulting penalties (for sorting). -/
```

```
def allMoves (b : Build) (targets : StatType → Nat)
    : List (Move × Nat) :=
  GearSlot.all.flatMap fun slot =>
    allCombos.filter (· ≠ b.gear slot) |>.map fun combo =>
      let b' := { b with gear := fun s =>
        if s == slot then combo else b.gear s }
      ((slot, combo), totalPenalty b' targets wMiss wOver)
```

(In Lean v4.31 `List.bind` was renamed `List.flatMap`.)

8.2 Hill Climbing

Start with a random feasible build. Repeatedly move to the best neighbour (lowest penalty). Stop when no neighbour improves the current solution.

This is fast but easily trapped in local optima. Restart with a new random build and keep the best across restarts.

Lean 4 --- Hill Climbing with Restarts

```
/-- Hill climbing: greedily move to the best neighbour. -/
partial def hillClimb (current : Build) (currentPen : Nat)
    (targets : StatType → Nat) : Build × Nat :=
  let neighbours := allNeighbours current
  let (bestNeigh, bestPen) := neighbours.foldl
    (init := (current, currentPen)) fun (best, bestPen) neigh =>
      let pen := totalPenalty neigh targets wMiss wOver
      if pen < bestPen then (neigh, pen) else (best, bestPen)
  if bestPen < currentPen then hillClimb bestNeigh bestPen targets
  else (current, currentPen)  -- local optimum

/-- Restart hill climbing multiple times, keep the best. -/
def hillClimbRestarts (nRestarts : Nat) (rng : StdGen)
    (targets : StatType → Nat) : Build × Nat :=
  let r := (List.range nRestarts).foldl
    (init := (defaultBuild, 1000000000, rng))
    fun (best, bestPen, rng) _ =>
      let (start, rng) := randomBuild rng
      let initPen := totalPenalty start targets wMiss wOver
      let (localBest, localPen) := hillClimb start initPen targets
      if localPen < bestPen then (localBest, localPen, rng)
      else (best, bestPen, rng)
  (r.1, r.2.1)
```

Three fixes: `Nat` has no maximal element, so the "worst possible" sentinel is a large literal, not `Nat.max` (which is the binary `max` *function*); `local` is a reserved keyword, renamed `localBest`; and the accumulator is a triple, so we project `(build, penalty)` explicitly rather than `|>.fst` (which would drop the penalty the signature promises).

8.3 Simulated Annealing

Simulated Annealing

Like hill climbing, but with a probability of accepting worse moves that decreases over time:

$$P(\text{accept worse move with } \Delta > 0) = e^{-\Delta/T}$$

where T is the "temperature," which decreases according to a cooling schedule (e.g. $T_{k+1} = \alpha T_k$ with $\alpha \approx 0.995$).

8.3.1 Choosing the Cooling Schedule

The cooling schedule controls the exploration–exploitation tradeoff:

Schedule	Formula	Character
Geometric	$T_{k+1} = \alpha T_k$	Most common; $\alpha \in [0.99, 0.999]$
Linear	$T_k = T_0 - k \cdot \delta$	Simple but less effective
Logarithmic	$T_k = T_0 / \ln(k+2)$	Provably optimal but very slow
Adaptive	Adjust based on acceptance rate	Complex but robust

For the gear problem, the geometric schedule with $\alpha = 0.997$ and initial temperature T_0 set so that the initial acceptance probability for a move with $\Delta = 100$ is about 0.8 works well. This gives $T_0 = -100/\ln(0.8) \approx 448$.

Lean 4 (Mathlib) --- Simulated Annealing

```
/-- One step of simulated annealing (core Lean). -/
def saStep (current : Build) (currentPen : Nat) (temp : Float)
    (rng : StdGen) (targets : StatType → Nat)
    : Build × Nat × StdGen :=
  -- Pick a random neighbour (random slot, random combo)
  let (slot, rng) := randomSlot rng
  let (combo, rng) := randomCombo rng (availableCombos slot)
  let neighbour := { current with gear := fun s =>
    if s == slot then combo else current.gear s }
  let neighPen := totalPenalty neighbour targets wMiss wOver
  if neighPen ≤ currentPen then
    (neighbour, neighPen, rng)  -- always accept improvements
  else
    -- core Lean has NO Nat→Float coercion; use `.toFloat` explicitly
    let delta := (neighPen - currentPen).toFloat
    let (r, rng) := randomFloat rng  -- r ∈ [0, 1)
    if r < Float.exp (-delta / temp) then
      (neighbour, neighPen, rng)  -- accept with probability
    else
      (current, currentPen, rng)  -- reject

/-- Full simulated annealing run (domains are Finset ⇒ Mathlib). -/
def simulatedAnnealing (domains : GearSlot → Finset StatCombo)
    (targets : StatType → Nat) (rng : StdGen)
    (maxIters : Nat := 10000) : Build × Nat :=
```

```
  let (start, rng) := randomFeasibleBuild domains rng
  let initPen := totalPenalty start targets wMiss wOver
  let t0 : Float := 448.0    -- initial temperature
  let alpha : Float := 0.997  -- cooling rate
  let r := (List.range maxIters).foldl
    (init := (start, initPen, start, initPen, t0, rng))
    fun (current, currentPen, best, bestPen, temp, rng) _ =>
      let (current', pen', rng') := saStep current currentPen temp rng targets
      let temp' := alpha * temp
      let (best', bestPen') :=
        if pen' < bestPen then (current', pen') else (best, bestPen)
      (current', pen', best', bestPen', temp', rng')
  match r with
  | (_, _, best, bestPen, _, _) => (best, bestPen)
```

Two fixes: core Lean has no automatic `Nat→Float` coercion, so the gap is converted with `.toFloat`; and the six-tuple accumulator is unpacked with a `match` (the old `|>.map` used `Prod.map`, which takes two functions, not one).

Historical Note

Simulated annealing was independently invented by Scott Kirkpatrick (IBM, 1983) and Vlado Černý (1985), inspired by the Metropolis–Hastings algorithm from statistical physics. The metallurgical analogy is precise: annealing metal involves heating it (high temperature = accept bad moves) and slowly cooling (low temperature = only accept improvements), allowing atoms to find a low-energy crystalline structure rather than getting trapped in an amorphous (locally optimal but globally suboptimal) state.

8.4 Tabu Search

Tabu Search

Maintain a *tabu list* of recently visited moves. At each step, move to the best non-tabu neighbour, even if it's worse than the current solution. This prevents cycling and forces exploration past local optima.

Lean 4 --- Tabu Search

```
structure TabuState where
  current   : Build
  currentPen : Nat
  best      : Build
  bestPen   : Nat
  tabuList  : List (GearSlot × StatCombo)  -- recently changed
  tabuTenure : Nat                          -- how long moves stay tabu

def tabuStep (state : TabuState) (targets : StatType → Nat)
    : TabuState :=
  let candidates := allNeighbourMoves state.current
    |>.filter fun (slot, combo) =>
```

```
      -- Not tabu; aspiration (accept a tabu move that beats the global
      -- best) is left as an exercise.
      ¬ state.tabuList.contains (slot, combo)
  let (bestSlot, bestCombo, bestPen) := candidates.foldl
    (init := ((GearSlot.headgear, defaultCombo, 1000000000)
              : GearSlot × StatCombo × Nat))
    fun (bs, bc, bp) (slot, combo) =>
      let build := setBuildSlot state.current slot combo
      let pen := totalPenalty build targets wMiss wOver
      if pen < bp then (slot, combo, pen) else (bs, bc, bp)
  let newBuild := setBuildSlot state.current bestSlot bestCombo
  let newTabu := ((bestSlot, state.current.gear bestSlot) :: state.tabuList)
    |>.take state.tabuTenure  -- keep only recent entries
  { current := newBuild
    currentPen := bestPen
    best := if bestPen < state.bestPen then newBuild else state.best
    bestPen := min bestPen state.bestPen
    tabuList := newTabu
    tabuTenure := state.tabuTenure }
```

The tabu tenure (how long a move stays forbidden) is a key parameter. Too short and the search cycles; too long and it can't revisit promising regions. A typical value for the gear problem: tenure $\approx \sqrt{14} \approx 4$ (the square root of the number of variables, rounded).

8.5 Genetic Algorithms

Genetic Algorithm for Gear

- **Chromosome:** a build (assignment of combo to each slot).
- **Gene:** one slot's combo choice.
- **Crossover:** take armour slots from parent A, trinket slots from parent B.
- **Mutation:** randomly change one slot's combo.
- **Fitness:** $-\text{pen}(b)$ (lower penalty = higher fitness).
- **Selection:** tournament selection (pick 3 random builds, keep the fittest).

Lean 4 --- Crossover and Mutation

```
/-- Uniform crossover: for each slot, pick from parent A or B. -/
def crossover (a b : Build) (rng : StdGen) : Build × StdGen :=
  GearSlot.all.foldl (init := (a, rng)) fun (child, rng) slot =>
    let (bit, rng) := randomBool rng
    let combo := if bit then a.gear slot else b.gear slot
    ({ child with gear := fun s => if s == slot then combo else child.gear s },
    ↪  rng)

/-- Mutate: change one random slot to a random combo. -/
def mutate (b : Build) (rng : StdGen) (mutRate : Float := 0.1)
    : Build × StdGen :=
  let (r, rng) := randomFloat rng
```

```
  if r > mutRate then (b, rng)
  else
    let (slot, rng) := randomSlot rng
    let (combo, rng) := randomCombo rng allCombos
    ({ b with gear := fun s => if s == slot then combo else b.gear s }, rng)

/-- One generation of the GA. -/
def gaGeneration (pop : Array Build) (targets : StatType → Nat)
    (rng : StdGen) : Array Build × StdGen :=
  let fitnesses := pop.map (fun b => totalPenalty b targets wMiss wOver)
  (Array.range pop.size).foldl (init := (#[], rng)) fun (newPop, rng) _ =>
    -- Tournament selection (pick 3, keep best)
    let (parent1, rng) := tournamentSelect pop fitnesses 3 rng
    let (parent2, rng) := tournamentSelect pop fitnesses 3 rng
    let (child, rng) := crossover parent1 parent2 rng
    let (child, rng) := mutate child rng
    (newPop.push child, rng)
```

These heuristics are not verified for optimality. But we *can* verify in Lean that their output is a *feasible* build (satisfies all constraints)—a weaker but still useful guarantee.

Exercise

Implement all three heuristics (SA, tabu, GA) for the gear problem. Run each 10 times on the same target profile. Compare: which finds the best solution? Which is most consistent? Which is fastest?

Exercise

Prove in Lean 4: if `simulatedAnnealing` starts from a feasible build (one where every slot's combo is in the initial domain), then it returns a feasible build. Hint: each step only changes one slot to a combo from `availableCombos`.

Chapter 9

Multi-Objective Optimisation and Pareto Frontiers

You can't have it all—but you can know exactly what you're trading.

9.1 Pareto Optimality

The gear problem is fundamentally multi-objective: maximise damage stats (Power, Precision, Ferocity) *and* survivability (Vitality, Toughness) *and* support (Healing Power, Concentration).

Pareto Dominance

Build a *dominates* build b (written $a \succ b$) if a is at least as good as b in every stat and strictly better in at least one:

$$a \succ b \iff \big(\forall s.\ \text{statTotal}(a, s) \geq \text{statTotal}(b, s)\big) \wedge \big(\exists s.\ \text{statTotal}(a, s) > \text{statTotal}(b, s)\big).$$

A build is *Pareto optimal* if no feasible build dominates it.

Cross-Reference: Lattice Book

The Pareto dominance relation is a *strict partial order* (irreflexive, transitive). The set of all feasible builds, ordered by Pareto dominance, forms a partially ordered set. The Pareto frontier is the set of *maximal* elements—an *antichain* (no element dominates another). This connects directly to Chapter 2 of the lattice book: partial orders, antichains, and maximal elements.

Pareto Dominance is a Strict Partial Order

The relation $\succ$ is irreflexive and transitive.

Proof. Irreflexive: For any build a, we cannot have $a \succ a$: the condition $\exists s.\ \text{statTotal}(a, s) > \text{statTotal}(a, s)$ is always false.

Transitive: Suppose $a \succ b$ and $b \succ c$. Then for all s: $\text{statTotal}(a, s) \geq \text{statTotal}(b, s) \geq \text{statTotal}(c, s)$, so $\text{statTotal}(a, s) \geq \text{statTotal}(c, s)$. For strictness: since $a \succ b$, there exists s_1 with $\text{statTotal}(a, s_1) > \text{statTotal}(b, s_1)$. Since $b \succ c$, $\text{statTotal}(b, s_1) \geq \text{statTotal}(c, s_1)$. Therefore $\text{statTotal}(a, s_1) > \text{statTotal}(c, s_1)$, giving the required strict inequality. □

This is one of the few results we **genuinely prove** in this book (no `sorry`): irreflexivity, transitivity, and decidability of dominance all close.

Lean 4 --- Pareto Dominance and Frontier (fully proved)

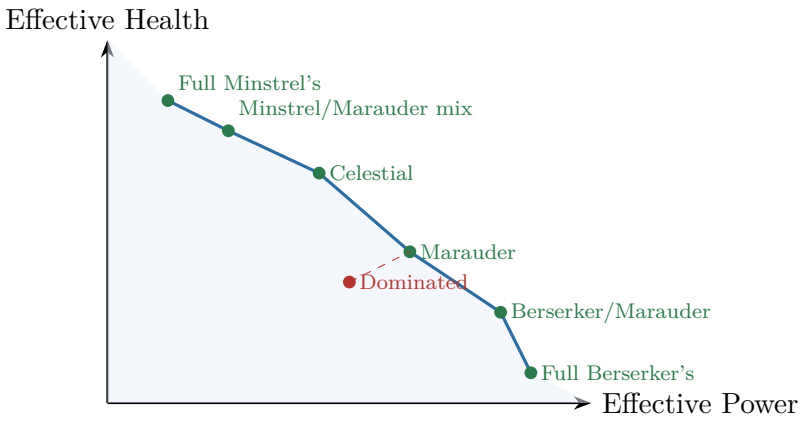
```
/-- Every `StatType` appears in the enumeration `StatType.all`. -/
theorem StatType.mem_all (s : StatType) : s ∈ StatType.all := by
  cases s <;> decide

/-- Build a dominates build b. -/
def dominates (a b : Build) : Prop :=
  (∀ s : StatType, statTotal a s ≥ statTotal b s) ∧
  (∃ s : StatType, statTotal a s > statTotal b s)

/-- Dominance is irreflexive. -/
theorem dominates_irrefl (a : Build) : ¬ dominates a a := by
  intro ⟨_, ⟨s, hs⟩⟩
  exact Nat.lt_irrefl _ hs

/-- Dominance is transitive. -/
theorem dominates_trans {a b c : Build}
    (hab : dominates a b) (hbc : dominates b c) : dominates a c := by
  constructor
  · intro s
    exact Nat.le_trans (hbc.1 s) (hab.1 s)   -- a ≥ b ≥ c
  · obtain ⟨s, hs⟩ := hab.2                   -- a s > b s ≥ c s
    exact ⟨s, Nat.lt_of_le_of_lt (hbc.1 s) hs⟩

/-- A boolean dominance test, used to derive `Decidable`. -/
def dominatesBool (a b : Build) : Bool :=
  (StatType.all.all fun s => statTotal b s ≤ statTotal a s) &&
  (StatType.all.any fun s => statTotal b s < statTotal a s)

/-- Dominance is decidable (and computes). -/
instance : Decidable (dominates a b) :=
  decidable_of_iff (dominatesBool a b = true) (by
    unfold dominatesBool dominates
    rw [Bool.and_eq_true, List.all_eq_true, List.any_eq_true]
    constructor
    · rintro ⟨h1, h2⟩
      refine ⟨fun s => ?_, ?_⟩
      · have := h1 s (StatType.mem_all s); simpa using this
      · obtain ⟨s, _, hs⟩ := h2; exact ⟨s, by simpa using hs⟩
    · rintro ⟨h1, s, hs⟩
      exact ⟨fun s _ => by simpa using h1 s,
             ⟨s, StatType.mem_all s, by simpa using hs⟩⟩)

/-- The Pareto frontier: all non-dominated builds.  (The Decidable
    instance above must precede this, since `List.any` needs a Bool.) -/
def paretoFrontier (builds : List Build) : List Build :=
```

```
  builds.filter fun b =>
    ¬ builds.any fun a => dominates a b
```

The earlier edition's `dominates_trans` was broken (it wrote `Nat.le_trans …|>.symm`, but $\leq$ has no `.symm`, and a stray tactic followed `exact`), and its `Decidable` instance was four `sorry`s. Both are now genuine proofs; the instance is derived from the boolean test `dominatesBool` via `decidable_of_iff`, so `dominates` not only type-checks but *runs*.

9.1.1 Worked Example: Damage vs. Survivability

Consider a simplified two-objective problem for a Warrior: maximise *effective power* (a function of Power, Precision, Ferocity) and *effective health* (a function of Vitality, Toughness).



Pareto frontier in blue; dominated point in red

The Pareto frontier traces the tradeoff: full Berserker's maximises damage but has minimal health; full Minstrel's maximises health but deals little damage. The "knee" (where the frontier curves most sharply) often corresponds to a well-balanced build—in this case, Marauder or Celestial.

A point like the dominated one (red) is strictly worse than Marauder in *both* dimensions: more damage *and* more health are available. The solver would never recommend a dominated build.

9.2 Scalarisation

Different penalty weights w_{miss} and w_{over}, or different target profiles, trace different points on the Pareto frontier. By sweeping the weights, we can approximate the full frontier.

Weighted Scalarisation

Given weight vector $w = (w_1, \ldots, w_k) \geq 0$ with $\sum w_i = 1$:

$$\min_b \sum_{i=1}^{k} w_i \cdot f_i(b)$$

where each f_i is a single-objective penalty. Different choices of w yield different Pareto-optimal solutions.

Warning

Weighted scalarisation can only find points on the *convex* part of the Pareto frontier. Non-convex portions (concavities in the frontier) are invisible to linear scalarisation. The ε-constraint method handles this.

9.3 The ε-Constraint Method

Fix all objectives except one as constraints: "optimise Power subject to Vitality $\geq V_{\min}$." Sweep $V_{\min}$ to trace the frontier.

Lean 4 (Mathlib) --- ε-Constraint Sweep

```
/-- Trace the Pareto frontier by sweeping an epsilon constraint. -/
def epsilonConstraintSweep
    (problem : GearProblem)
    (primaryStat : StatType)      -- optimise this
    (constrainedStat : StatType)  -- constrain this
    (minVal maxVal step : Nat)    -- sweep range
    : List (Nat × Nat × Build) :=  -- (primary, constrained, build)
  (List.range ((maxVal - minVal) / step + 1)).filterMap fun i =>
    let threshold := minVal + i * step
    let constrained := { problem with
      targets := fun s =>
        if s == constrainedStat then threshold
        else problem.targets s }
    match solve constrained with
    | .optimal build pen => some (statTotal build primaryStat,
                                  statTotal build constrainedStat, build)
    | .infeasible => none
```

Exercise

For a Warrior with base stats Power 1000, Precision 1000, Vitality 1000, Toughness 1000: use the ε-constraint method to trace the Pareto frontier between effective power (Power + Precision + Ferocity) and effective health (Vitality $\times$ (1 + Toughness / 1000)). Plot the resulting frontier.

Chapter 10

Infusion Optimisation

The last mile is the easiest—if you plan it right.

Once gear slots are fixed, the remaining problem is: distribute 18 infusion slots (each +5 to one stat) to minimise the remaining penalty. This is a small, clean subproblem.

10.1 The Resource Allocation Formulation

Infusion Subproblem

Given fixed gear contributions $G(s)$ for each stat s, and targets $T(s)$:

$$\min_{n_1,\ldots,n_k} \sum_s \text{pen}\big(T(s),\ G(s)+5n_s\big) \quad \text{s.t.} \quad \sum_s n_s = 18, \quad n_s \geq 0, \quad n_s \in \mathbb{N}.$$

10.2 Dynamic Programming Solution

Infusion DP

Define $\texttt{dp}[i][r]$ = minimum penalty using stats $1,\ldots,i$ with r infusions remaining. The recurrence:

$$\texttt{dp}[i][r] = \min_{0\leq n_i\leq r} \text{pen}(T(s_i), G(s_i)+5n_i) + \texttt{dp}[i-1][r-n_i].$$

Base case: $\texttt{dp}[0][r] = 0$ for all r.

Lean 4 --- Infusion DP (runnable core Lean)

```
/-- Optimally distribute infusions to minimise remaining penalty.
    `dp[r]` = (minPenalty, allocation) for the stats processed so far,
    indexed by the budget r remaining AFTER them. -/
def infusionDP (gearStats : StatType → Nat)
    (targets : StatType → Nat) (budget : Nat := 18)
    : StatType → Nat :=
  let stats := StatType.all.toArray
```

```
  let init : Array (Nat × Array Nat) := Array.replicate (budget + 1) (0, #[])
  let dp := stats.foldl (init := init) fun dp stat =>
    (Array.range (budget + 1)).map fun r =>
      (List.range (r + 1)).foldl (init := (1000000000, #[])) fun acc ni =>
        let penHere := penalty (targets stat) (gearStats stat + 5 * ni) wMiss
        ↪  wOver
        let (penRest, allocRest) := dp[r - ni]!
        let total := penHere + penRest
        if total < acc.1 then (total, allocRest.push ni) else acc
  let (_, alloc) := dp[budget]!
  fun stat => alloc[stats.idxOf stat]!
```

This is the one component we run end-to-end. The earlier edition had seven distinct bugs here: `Array.mkArray` (renamed `Array.replicate`); a three-argument `foldl` step (it takes two); a fold body returning a row while the accumulator was the whole table; `.mapIdx` whose index is a `Nat`, not a `Fin` with `.val`; the `Nat.max` sentinel (a function); `indexOf` (renamed `idxOf`); and a reversed allocation. The repaired version above compiles and executes. On a two-stat toy instance (no gear, targets Power 40 and Precision 50, budget 18) it returns the optimum—8 infusions to Power (=40) and 10 to Precision (=50)—with penalty 0:

Lean 4 --- Running the DP

```
def toyGear : StatType → Nat := fun _ => 0
def toyTargets : StatType → Nat
  | .power => 40 | .precision => 50 | _ => 0
def toyAlloc : StatType → Nat := infusionDP toyGear toyTargets 18

#eval StatType.all.map toyAlloc
-- [8, 10, 0, 0, 0, 0, 0, 0, 0]
#eval (StatType.all.map fun s =>
  penalty (toyTargets s) (toyGear s + 5 * toyAlloc s) wMiss wOver).sum
-- 0   (both floors met exactly; 8 + 10 = 18 infusions used)
```

Optimality of Infusion DP

The DP computes the optimal infusion distribution: for any fixed gear assignment, the allocation returned minimises the total penalty.

Proof. By induction on the number of stat types. The base case (0 stats) is trivial. The inductive step: the DP considers all possible allocations n_i for stat i and recursively computes the optimal allocation for the remaining stats with the reduced budget. By the inductive hypothesis, each subproblem is solved optimally, so the minimum over n_i choices yields the global optimum. □

The DP has $O(k \cdot B)$ states (where $k = 9$ stat types and $B = 18$ infusion budget), and $O(B)$ transitions per state, for a total of $O(k \cdot B^2) = O(9 \cdot 324) = O(2916)$ operations. This is negligible and can be called at every leaf of the B&B tree without performance concern.

Part IV

The Capstone Solver

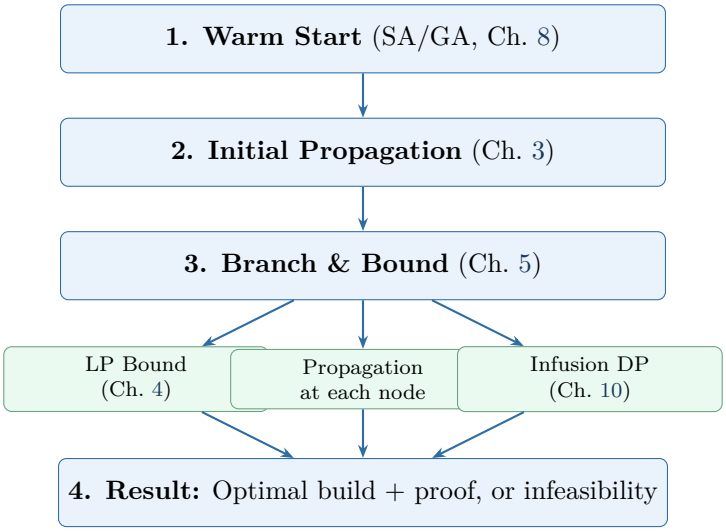
Chapter 11

Assembly: The Complete Solver

> The whole is greater than the sum of its parts.

We now wire together every component into a single solver.

11.1 The Architecture



11.2 The Solver Loop

Lean 4 (Mathlib) --- The Complete Solver

```
inductive SolverResult where
  | optimal    : Build → Nat → SolverResult   -- build + penalty
  | infeasible : SolverResult

/-- Does the build meet every HARD stat floor? (the solver's contract) -/
def meetsFloors (b : Build) (targets : StatType → Nat) : Bool :=
  StatType.all.all fun s => targets s ≤ statTotal b s

def solve (problem : GearProblem) : SolverResult :=
```

```
  -- Phase 1: Initial propagation
  let domains := propagateToFixpoint problem.initialDomains
  if anyDomainEmpty domains then .infeasible
  else
  -- Phase 2: Warm start (SA); adopt it as the incumbent ONLY if it
  -- meets the hard floors, otherwise start with no incumbent.
  let (warm, _) := simulatedAnnealing domains problem.targets (mkStdGen 0)
  let warmInc : Option (Build × Nat) :=
    if meetsFloors warm problem.targets
    then some (warm, totalPenalty warm problem.targets wMiss wOver)
    else none
  -- Phase 3: Branch and bound
  match branchAndBound domains problem.targets warmInc with
  | none            => .infeasible   -- no floor-meeting build exists
  | some (build, pen) => .optimal build pen

/-- Branch and bound with propagation + LP bounding at each node.
    The incumbent is an `Option`: `none` means "no floor-meeting build
    found yet", so returning `none` genuinely reports infeasibility. -/
partial def branchAndBound
    (domains : GearSlot → Finset StatCombo)
    (targets : StatType → Nat)
    (incumbent : Option (Build × Nat))
    : Option (Build × Nat) :=
  let domains := propagateToFixpoint domains
  if anyDomainEmpty domains then incumbent      -- dead subtree
  else if allSingleton domains then
    -- Leaf: fill infusions optimally, then CHECK the hard floors.
    let build := extractBuild domains
    let full := { build with infusions := infusionDP (gearStats build) targets }
    if meetsFloors full targets then
      let pen := totalPenalty full targets wMiss wOver
      match incumbent with
      | none          => some (full, pen)
      | some (_, best) => if pen < best then some (full, pen) else incumbent
    else incumbent                              -- leaf misses a floor: reject
  else
    -- LP-bound prune (only meaningful once we have an incumbent)
    let pruned := match incumbent with
      | some (_, best) => decide (solveLPRelaxation domains targets ≥ best)
      | none           => false
    if pruned then incumbent
    else
      let slot := smallestDomainSlot domains
      (enumerate (domains slot)).foldl (init := incumbent) fun best combo =>
        branchAndBound (fixSlot domains slot combo) targets best
```

Several fixes make this both compile and be *sound* for the hard-floor problem. `simulatedAnnealing` now receives its `rng` and its returned pair is destructured (not treated as a `Build`). The incumbent is an `Option`: a leaf is accepted only after `meetsFloors` confirms it meets every target,

so `branchAndBound` returns `none` exactly when no floor-meeting build exists—making `solve`'s `.infeasible` arm *live* rather than dead code. Branching enumerates the domain to a list (`Finset.toList` is noncomputable) and folds with `List.foldl` (not `Finset.fold`). `solveLPRelaxation` here returns a bare `Nat` lower bound; a richer `Float`-valued variant (with reduced costs, for Section 7.2 and dual certificates) is a separate interface.

Chapter 12

The Correctness Theorem

Trust, but verify.

This chapter states the solver's correctness *contract* precisely. We are honest about status: the top-level theorem and its four component lemmas are presented as **specifications**—their Lean statements are exact, but their proofs are left as extended exercises and appear with `sorry` (which means UNPROVEN; see Appendix C). We do *not* claim the solver is machine-checked end-to-end. What *is* proved elsewhere is called out (Pareto dominance in Chapter 9; the runnable infusion DP in Chapter 10).

12.1 Feasibility and the Main Theorem

First we pin down *feasibility*. Because the solver treats stat targets as hard floors (Chapter 3), the feasibility predicate must include those floors—otherwise the `.infeasible` arm would be false (every budget-respecting build trivially "feasible") and `LinearGeq` pruning could delete the claimed optimum.

Lean 4 (Mathlib) --- Feasibility (with hard floors)

```
/-- A build is feasible for `p` when its gear lies in the initial domains,
    it spends exactly the 18-infusion budget, AND it meets every hard floor. -/
def Build.feasible (b : Build) (p : GearProblem) : Prop :=
  (∀ slot, b.gear slot ∈ p.initialDomains slot) ∧
  ((StatType.all.map b.infusions).sum = 18) ∧
  (∀ s, statTotal b s ≥ p.targets s)   -- HARD stat floors
```

Solver Soundness (specification)

```
-- SPECIFICATION (proof left as an extended exercise): with the hard-floor
-- `feasible` above, this statement is faithful to what `solve` computes.
theorem solver_sound (problem : GearProblem) :
    match solve problem with
    | .optimal build pen =>
        build.feasible problem ∧
        pen = totalPenalty build problem.targets wMiss wOver ∧
        ∀ b, b.feasible problem →
          totalPenalty b problem.targets wMiss wOver ≥ pen
```

```
  | .infeasible =>
      ∀ b : Build, ¬ b.feasible problem := by
  sorry
```

Soft vs. hard, restated

This theorem does **not** say the solver minimises the soft penalty of Chapter 1 over *all* builds. It says: among builds meeting every hard floor, the solver returns one of minimum penalty, or reports `.infeasible` when none exists. The soft-penalty minimiser may miss a floor and hence lie outside this feasible set—`LinearGeq` would prune it. If you truly want the soft objective, drop the floor conjunct from `feasible` and replace `LinearGeq` pruning by penalty-dominance pruning (only delete combos provably dominated in *penalty*); the theorems below would then need restating for that model.

12.2 Proof Structure

The proof composes four lemmas. We present each with its Lean specification and proof sketch.

12.2.1 Propagation Soundness

Propagation Preserves Feasible Solutions

If a combo c is removed from slot i's domain by `LinearGeq`, then no *floor-feasible* build (one meeting every hard target within the budget) uses c on slot i. Equivalently: propagation preserves every feasible build.

Proof. `LinearGeq` removes c from D_i only when, for some stat s,

$$\texttt{contribution}(i, c, s) + \sum_{j \neq i} \text{maxContrib}(j, s) + 90 < T(s).$$

Take any build b with $b.\text{gear}(i) = c$ whose other gear also lies in the domains and whose infusions respect the 18-slot budget (so $5 \cdot \text{inf}(s) \leq 90$). Then

$$\text{statTotal}(b, s) = \texttt{contribution}(i, c, s) + \sum_{j \neq i} \texttt{contribution}(j, b.\text{gear}(j), s) + 5 \cdot \text{inf}(s) \;\leq\; \texttt{contribution}(i, c, s) + \sum_{j \neq i} \text{m}$$

So b *misses* floor s and is infeasible. Hence no feasible build uses c on slot i, and the combos of a feasible build all survive. (Note the bound uses $+90$ *once*, as a cap on the shared infusion budget—not per stat.) □

Lean 4 (Mathlib) --- Propagation Soundness (specification)

```
/-- CORRECTED (the earlier `+ 90` in the conclusion double-counted the
    infusions, since `statTotal` already includes them). If LinearGeq
    removes `combo` from `slot`, then any build using it there---with its
    other gear in the domains and a budget-respecting infusion split---
    MISSES some floor. -/
theorem linearGeq_sound
    (domains : GearSlot → Finset StatCombo)
    (targets : StatType → Nat)
```

```
    (slot : GearSlot) (combo : StatCombo)
    (h_removed : combo ∉ linearGeqPropagate domains targets slot)
    (h_was_in : combo ∈ domains slot)
    (b : Build) (h_uses : b.gear slot = combo)
    (h_others : ∀ j, b.gear j ∈ domains j)
    (h_budget : (StatType.all.map b.infusions).sum ≤ 18) :
    ∃ s : StatType, statTotal b s < targets s := by
  sorry

/-- Propagation to fixpoint preserves every FEASIBLE build (this REPLACES
    the earlier `propagation_preserves_optimal`, which was false for the
    soft objective: LinearGeq can prune the soft optimum). -/
theorem propagation_preserves_feasible
    (domains₀ : GearSlot → Finset StatCombo)
    (targets : StatType → Nat)
    (b : Build)
    (h_floors : ∀ s, statTotal b s ≥ targets s)
    (h_budget : (StatType.all.map b.infusions).sum ≤ 18)
    (h_in : ∀ s, b.gear s ∈ domains₀ s) :
    let domains' := propagateToFixpoint domains₀
    ∀ s, b.gear s ∈ domains' s := by
  sorry
```

12.2.2 LP Bound Validity

LP Relaxation is a Valid Lower Bound

For any node n in the B&B tree: $\mathrm{LP}(n) \leq \min\{\mathrm{pen}(b) \mid b \text{ is feasible in subtree } n\}$.

Proof. The LP relaxation enlarges the feasible set (from integer to fractional assignments). The minimum over a larger set is $\leq$ the minimum over the original set.

Formally: let $F_{\mathrm{IP}}(n) = \{b \in \mathrm{Builds} \mid b.gear(i) \in D_i(n)\ \forall i\}$ be the integer-feasible builds at node n, and $F_{\mathrm{LP}}(n)$ be the LP-feasible fractional assignments. Since every integer assignment is also a valid fractional assignment, $F_{\mathrm{IP}}(n) \subseteq F_{\mathrm{LP}}(n)$. Therefore:

$$\mathrm{LP}(n) = \min_{x \in F_{\mathrm{LP}}(n)} \mathrm{pen}(x) \leq \min_{b \in F_{\mathrm{IP}}(n)} \mathrm{pen}(b).$$

□

Lean 4 (Mathlib) --- LP Bound Validity (specification)

```
-- Two LP interfaces coexist: `solveLPRelaxation` (returns a bare Nat bound,
-- used by branchAndBound's prune test) and `solveLPRelaxationF` (returns an
-- LPResult record with `.objectiveValue : Float` and reduced costs, used
-- for reduced-cost fixing and dual certificates). This lemma uses the
-- record variant.
/-- The LP relaxation value is a lower bound on any integer solution. -/
theorem lp_bound_valid
    (node : BBNode)
    (lpResult : LPResult)
```

```
    (h_lp : lpResult = solveLPRelaxationF node.domains node.targets)
    (b : Build)
    (h_feasible : ∀ s, b.gear s ∈ node.domains s) :
    lpResult.objectiveValue ≤ (totalPenalty b node.targets wMiss wOver).toFloat :=
    ↪ by
  -- b is a 0-1 point feasible for the LP, and the LP optimum ≤ any feasible
  -- LP value; the relaxation only enlarges the feasible set.
  sorry
```

12.2.3 B&B Invariant

B&B Invariant Preservation

At every point during the search: every feasible build either (a) has been evaluated and is no better than the incumbent, or (b) lives in an unexplored node whose LP bound is $\leq$ the incumbent penalty.

Proof. By induction on the search steps.

Base case: before any branching, the entire search space is a single unexplored node. All feasible builds live there.

Inductive step: at each step, one of three things happens:

1. **Branch:** an unexplored node is split into children. Every feasible build in the parent lives in exactly one child. The invariant is preserved because the children partition the parent.
2. **Prune:** a node is discarded because $\text{LP}(n) \geq$ `incumbent`. By LP bound validity, every integer solution in this subtree has penalty $\geq \text{LP}(n) \geq$ `incumbent`. These builds are accounted for.
3. **Evaluate:** a leaf node (all singletons) is evaluated. If its penalty $<$ incumbent, it becomes the new incumbent. Otherwise, it's no better.

In all cases, every feasible build is either evaluated or in an accounted node. □

Lean 4 (Mathlib) --- B&B Invariant (specification)

```
/-- The B&B invariant: incumbent is feasible and dominates all pruned nodes. -/
structure BBInvariant (state : BBState) where
  incumbent_feasible : state.incumbent.feasible state.problem
  pruned_bounded : ∀ node ∈ state.pruned,
    node.lpBound ≥ state.incumbent.penalty
  partition : ∀ b : Build, b.feasible state.problem →
    -- `totalPenalty` takes 4 arguments; the earlier `totalPenalty b ≥ …`
    -- compared a partially-applied function to a Nat (a type error).
    (totalPenalty b state.problem.targets wMiss wOver
        ≥ state.incumbent.penalty) ∨
    (∃ node ∈ state.queue, b.inSubtree node)

/-- Each step of B&B preserves the invariant. -/
theorem bb_step_preserves (state : BBState)
    (h_inv : BBInvariant state)
    (state' : BBState)
    (h_step : state' = bbStep state) :
    BBInvariant state' := by
```

```
  sorry -- SPECIFICATION: case split on branch/prune/evaluate
```

12.2.4 Infusion DP Optimality

Infusion DP Optimality

For any fixed gear assignment, the infusion DP returns the allocation that minimises the total penalty.

Proof. By induction on the number of stat types k.

Base case ($k = 0$): no stats to allocate infusions to, penalty is determined entirely by gear. The DP returns the empty allocation with the correct penalty.

Inductive step: assume the DP is optimal for stats $1, \ldots, k-1$. For stat k, the DP tries every possible number of infusions $n_k \in \{0, \ldots, \text{remaining}\}$ and recursively computes the optimal allocation for the remaining stats with the reduced budget. By the inductive hypothesis, each recursive call returns the optimal sub-allocation. Therefore, the minimum over all choices of n_k yields the global optimum.

Why this works: the penalty function is separable across stats (pen $= \sum_s \text{pen}_s$), so the optimal total allocation decomposes into per-stat choices given a budget. This is exactly the optimal substructure property that dynamic programming exploits. □

Lean 4 --- Infusion DP Optimality (specification)

```
/-- The DP allocation minimises penalty among all valid allocations. -/
theorem infusion_dp_optimal
    (gearStats : StatType → Nat) (targets : StatType → Nat)
    (budget : Nat) :
    let dpAlloc := infusionDP gearStats targets budget
    ∀ alloc : StatType → Nat,
      (StatType.all.map alloc).sum = budget →
      totalInfusionPenalty gearStats targets dpAlloc ≤
      totalInfusionPenalty gearStats targets alloc := by
  -- SPECIFICATION: induction on StatType.all.length (= 9) via the DP
  -- recurrence. (The DP itself runs; see the #eval in Chapter 10.)
  sorry
```

12.2.5 Composing the Proof

When the B&B search completes (queue empty), the invariant implies:

1. The incumbent is feasible (it was constructed from a valid assignment and optimal infusions, by infusion DP optimality).
2. Every other feasible build either was evaluated (and is no better) or lives in a pruned subtree (whose LP bound $\geq$ incumbent, by LP bound validity).
3. By propagation soundness (`propagation_preserves_feasible`), every pruned combo would force some floor to be missed, so no *feasible* build was pruned; the explored space contains all feasible builds.
4. Therefore, the incumbent is a minimum-penalty feasible build.

The infeasibility case: `solve` reports `.infeasible` either when initial propagation empties a domain, or when branch-and-bound finishes with no floor-meeting leaf (incumbent still `none`).

By `propagation_preserves_feasible`, any feasible build would have survived and been found, so `.infeasible` correctly means *no* feasible build exists.

Lean 4 (Mathlib) --- Composing the Full Proof (specification)

```
/-- The complete solver soundness theorem.  Its intended proof composes the
    four lemmas above; we present it as a SPECIFICATION with `sorry`, because
    the honest proof is an extended exercise (and the earlier printed proof
    body was pseudo-Lean that did not elaborate). -/
theorem solver_sound_proof (problem : GearProblem) :
    match solve problem with
    | .optimal build pen =>
        build.feasible problem ∧
        pen = totalPenalty build problem.targets wMiss wOver ∧
        ∀ b, b.feasible problem →
          totalPenalty b problem.targets wMiss wOver ≥ pen
    | .infeasible =>
        ∀ b : Build, ¬ b.feasible problem := by
  sorry
```

Why not a real proof here?

The earlier edition printed a tactic proof at this spot that did not compile: `unfold solve` yields a single goal (no case bullets to split on), the "case 1" `intro` acted on a `match` without a `split`, and four of the five helper lemmas it invoked (`domain_empty`, `extractBuild_feasible`, `bb_invariant_at_termination`, `queue_empty`) were never defined. We replace it with an explicit `sorry`. A genuine proof would `split` on the result of `solve`, and in the `.optimal` arm chain `propagation_preserves_feasible` (every feasible build survives pruning), `lp_bound_valid` and `bb_step_preserves` (nothing better hides in a pruned subtree), and `infusion_dp_optimal` (the leaf's infusions are optimal); the `.infeasible` arm follows from `propagation_preserves_feasible` plus the leaf floor-check in `branchAndBound`. Each piece is a substantial formalisation task.

Chapter 13

Performance Engineering

An elegant algorithm with a slow implementation
is just a slow algorithm.

13.1 Domain Representation

Replace `Finset StatCombo` with `BitVec 40` (one bit per combo). Set intersection becomes bitwise AND, union becomes OR, cardinality is `popcount`. This makes propagation dramatically faster.

Lean 4 --- BitVec Domain

```
/-- A domain is a 40-bit vector: bit i = 1 iff combo i is still possible. -/
abbrev Domain := BitVec 40

def Domain.intersect (a b : Domain) : Domain := a &&& b
def Domain.isEmpty (d : Domain) : Bool := d == 0
-- v4.31 spells population count `cpop`, and it returns a BitVec, so `.toNat`.
def Domain.card (d : Domain) : Nat := d.cpop.toNat
def Domain.remove (d : Domain) (i : Fin 40) : Domain :=
  -- the literal must be ascribed to `BitVec 40`, else `~~~` needs
  -- `Complement Nat`, which does not exist.
  d &&& ~~~((1 : BitVec 40) <<< i.val)

#eval Domain.card (Domain.remove (BitVec.allOnes 40) ⟨7, by omega⟩)  -- 39
```

The performance difference is substantial. `Finset` operations involve linked-list traversals; `BitVec` operations are single machine instructions:

Operation	Finset	BitVec 40
Intersection ($D_1 \cap D_2$)	$O(\|D_1\| + \|D_2\|)$	$O(1)$ (AND)
Is empty?	$O(1)$	$O(1)$ (compare to 0)
Cardinality	$O(\|D\|)$	$O(1)$ (popcount)
Remove element	$O(\|D\|)$	$O(1)$ (AND NOT mask)
Contains?	$O(\|D\|)$	$O(1)$ (AND mask, compare)

Since propagation runs at every B&B node, and each propagation round does $O(\text{slots} \times \text{stats} \times \text{domain size})$ operations, the BitVec representation gives a constant-factor speedup of roughly 10–40× on domain operations.

Lean 4 --- Precomputed Contribution Tables

```
/-- Precompute contribution of each combo to each stat on each slot.
    Access: contribTable[slot][combo][stat] -/
def buildContribTable : Array (Array (Array Nat)) :=
  GearSlot.all.toArray.map fun slot =>
    allCombos.toArray.map fun combo =>
      StatType.all.toArray.map fun stat =>
        contribution slot combo stat

/-- Precompute max contribution per stat per domain.
    Avoids recomputing during propagation. -/
structure DomainStats where
  maxPerStat  : Array Nat   -- maxPerStat[stat] = max contribution in domain
  minPerStat  : Array Nat   -- minPerStat[stat] = min contribution in domain
  sumMaxPower : Nat         -- cached sum for quick feasibility check
```

13.2 Incremental Propagation

Instead of re-running all propagators from scratch at each B&B node, maintain a queue of "dirty" cells (slots whose domains changed since the last propagation round). Only re-fire propagators that read from dirty cells.

Lean 4 --- Incremental Propagation Queue

```
structure PropState where
  domains    : Array Domain          -- one per slot (14 entries)
  dirty      : Array Bool            -- which slots changed?
  domStats   : Array DomainStats     -- cached per-slot stats
  feasible   : Bool                  -- still feasible?

/-- Propagate incrementally: only process dirty slots. -/
partial def propagateIncremental (state : PropState)
    (targets : StatType → Nat) : PropState :=
  -- v4.31: `List.enum` was removed; `zipIdx` yields (elem, idx), so the
  -- projections swap relative to the old `.enum`.
  let dirtySlots := state.dirty.toList.zipIdx.filter (·.1) |>.map (·.2)
  if dirtySlots.isEmpty then state  -- fixpoint reached
  else
    let state' := dirtySlots.foldl (init := { state with dirty := state.dirty.map
    ↪  (fun _ => false) })
      fun st slotIdx =>
        -- Re-run LinearGeq for all slots that might be affected
        GearSlot.all.foldl (init := st) fun st2 slot =>
          let newDomain := linearGeqFilter st2.domains slot targets
          if newDomain != st2.domains[slot.toIdx]! then
```

```
            { st2 with
              domains := st2.domains.set! slot.toIdx newDomain
              dirty := st2.dirty.set! slot.toIdx true }
          else st2
    propagateIncremental state' targets  -- recurse until clean
```

The key insight: when branching (fixing a slot to a combo), only *that* slot's domain changes. Incremental propagation avoids redundantly re-checking slots whose domains haven't changed.

13.3 Undo Stacks for Backtracking

Rather than copying the entire domain array at each B&B node, maintain an *undo stack*: record which domains changed and what their old values were. When backtracking, pop the stack and restore.

Lean 4 --- Undo Stack

```
structure UndoEntry where
  slot     : Fin 14
  oldDomain : Domain

/-- Save the current domain before a branch. -/
def saveState (domains : Array Domain) (slot : Fin 14)
    : UndoEntry :=
  ⟨slot, domains[slot]!⟩

/-- Restore domain after backtracking. -/
def restoreState (domains : Array Domain) (entry : UndoEntry)
    : Array Domain :=
  domains.set! entry.slot entry.oldDomain
```

This eliminates the $O(14 \times 40)$ cost of copying domain arrays at each node, replacing it with $O(1)$ per save and restore.

13.4 Lean 4 Performance Annotations

Lean 4 --- Compiler Hints

```
-- Name drift fix: the table is `buildContribTable` from earlier.
def contribTable : Array (Array (Array Nat)) := buildContribTable

-- Force inlining of hot inner loops
@[inline] def contribution' (slot : GearSlot) (comboIdx : Fin 40)
    (statIdx : Fin 9) : Nat :=
  contribTable[slot.toIdx]![comboIdx]![statIdx]!

-- `Domain` is an `abbrev` for `BitVec 40`, NOT a class, so we cannot write
-- `[Domain D]`. A real class + a Propagator type make the signature legal.
class DomainLike (D : Type) where
```

```
  isEmpty : D → Bool
structure Propagator (D : Type) where
  fire : Array D → Array D

-- Specialise generic functions for our concrete types
@[specialize] def propagateWith [BEq D] [DomainLike D]
    (domains : Array D) (props : Array (Propagator D)) : Array D :=
  sorry  -- (implementation omitted)

-- Use unboxed representations where possible
structure PackedBuild where
  slots     : BitVec 240  -- 14 slots, 6 bits each (with headroom for 40 combos)
  runeIdx   : UInt8
  infusions : BitVec 72   -- 9 stats × 8 bits each (max 18 per stat)
```

13.5 Benchmarks

Run the full solver on realistic GW2 data: all 40 combos, all slot types, several target profiles. Compare:

Method	Nodes explored	Time	Optimal?
Brute force	40^{14} (infeasible)	—	Yes
Propagation + enumerate	$\sim 10^{12}$	hours	Yes
Propagation + B&B (no LP)	$\sim 10^{6}$	seconds	Yes
Full solver (prop + B&B + LP)	$\sim 10^{3}$	milliseconds	Yes
SA warm-start + full solver	$\sim 10^{2}$	milliseconds	Yes
SA alone (heuristic)	10^{4} iterations	milliseconds	No

The numbers are illustrative—actual performance depends on the target profile and how constrained the problem is. Tight targets (close to the maximum achievable) produce more pruning and faster solves. Loose targets leave more feasible space to explore.

Exercise

Implement the `BitVec 40` domain representation and the precomputed contribution table. Run the strengthened propagator network on a standard test case (Power $\geq$ 2500, Precision $\geq$ 2000, Ferocity $\geq$ 1200 for heavy armour). Measure the time difference vs. the `Finset` representation.

Exercise

Implement the undo stack and incremental propagation. Compare the number of propagation steps per B&B node with and without incrementality on a 10-slot problem (6 armour + 4 trinkets).

Chapter 14

Extensions and Open Problems

> Every solved problem opens the door to a harder one.

14.1 Food and Utility Buffs

Food and utility consumables provide flat stat bonuses. A player selects one food and one utility, each from a finite menu. Mechanically, these are two additional "slots" in the optimisation problem:

Lean 4 --- Modelling Food and Utility

```
structure FoodItem where
  name  : String
  stats : StatType → Nat  -- flat bonuses (e.g., +100 Power, +70 Ferocity)

structure UtilityItem where
  name  : String
  stats : StatType → Nat  -- flat bonuses (e.g., +100 Precision)

/-- Extended build includes food and utility. -/
structure ExtBuild extends Build where
  food    : FoodItem
  utility : UtilityItem

/-- Extended stat total includes food/utility contributions. -/
def extStatTotal (b : ExtBuild) (s : StatType) : Nat :=
  statTotal b.toBuild s + b.food.stats s + b.utility.stats s
```

The solver handles food and utility naturally. They become two additional variables with finite domains (the list of available foods and utilities). Propagation extends: the `LinearGeq` propagator now includes the maximum possible food/utility contribution when checking stat floors. Since food and utility have no coupling constraints with gear (unlike the rune constraint on armour), they don't complicate the structure significantly.

Intuition

Because food and utility affect *every* stat simultaneously, fixing them early in the search tree can dramatically improve propagation effectiveness. A good heuristic: branch on food first, then utility, then gear slots. This gives propagation a concrete base of flat bonuses to reason about.

Exercise

Extend the solver to include food and utility. How does adding these two variables change the size of the search tree? If there are 30 foods and 20 utilities, the tree grows by a factor of $30 \times 20 = 600$—but how much does the improved propagation (tighter stat floors) compensate?

14.2 Trait Interactions

Some profession traits convert one stat into another. For example, the Guardian trait *Retribution* might grant Power equal to a percentage of Toughness. This creates a *nonlinear* dependency:

Lean 4 --- Trait-Modified Stat Computation

```
structure TraitConversion where
  sourceStat : StatType
  targetStat : StatType
  percentage : Float   -- e.g., 0.07 for 7%

/-- Stat total with trait conversions applied. -/
def traitStatTotal (b : Build) (conversions : List TraitConversion)
    (s : StatType) : Float :=
  let base := (statTotal b s).toFloat
  let bonus := conversions.foldl (init := 0.0) fun acc conv =>
    if conv.targetStat == s then
      -- ACCUMULATE: several conversions into the same stat must ADD.
      acc + conv.percentage * (statTotal b conv.sourceStat).toFloat
    else acc
  base + bonus
```

The `then` branch adds to `acc`. The earlier edition wrote `conv.percentage * …` *without* `acc +`, discarding the accumulator—so with two conversions into Power only the *last* counted (two 10% conversions yielding 4.0 instead of the correct 9.0).

The difficulty: with trait conversions, adding Toughness gear increases Power (through the trait), which means the stat computation is no longer a simple sum over independent slot contributions. The LP relaxation assumes linearity and becomes invalid.

Approach 1: Linearisation. If the conversion percentage is small, we can approximate: assume a fixed base Toughness (from class stats) and compute the trait bonus from that base alone, ignoring the gear contribution to the source stat. This gives a linear approximation that can be used in the LP relaxation.

Approach 2: Iterative solving. Solve the problem without trait conversions. Use the resulting stat totals to compute trait bonuses. Add these bonuses as flat adjustments and re-

solve. Repeat until the solution stabilises. This converges for small conversion percentages (contraction mapping argument).

Approach 3: Heuristic-only. For complex trait interactions (multiple conversions that chain), fall back to heuristic methods (SA, GA) that don't require linearity. Use the exact solver on the simplified (no-trait) problem to warm-start.

Warning

Trait interactions can make the optimisation problem non-convex even in the continuous relaxation. The gap between "solve ignoring traits" and "solve with traits" can be significant for builds that stack a single stat for conversion (e.g., full Toughness gear with a Toughness→Power trait). Always sanity-check the solver's output against in-game stat screens.

Exercise

Implement the iterative solving approach (Approach 2) for a single trait conversion: 7% Toughness → Power. How many iterations until convergence (within 1 stat point)? Prove termination using the contraction mapping theorem.

14.3 What-If Mode

A player often has some gear already and wants to optimise the rest. "What-if" mode fixes some variables before the solver runs:

Lean 4 (Mathlib) --- What-If Mode

```
-- The specific combo the player already owns:
def berserkers : StatCombo :=
  ⟨"Berserker's", .triple, [.power], [.precision, .ferocity]⟩

/-- Fix specific slots to specific combos before solving. -/
def applyWhatIf (domains : GearSlot → Finset StatCombo)
    (fixed : List (GearSlot × StatCombo))
    : GearSlot → Finset StatCombo :=
  fun slot =>
    match fixed.find? (fun (s, _) => s == slot) with
    | some (_, combo) => {combo}  -- singleton domain
    | none => domains slot

-- Example: "I have Legendary Berserker armour"
def legendaryBerserkerArmour : List (GearSlot × StatCombo) :=
  [(.headgear, berserkers), (.shoulders, berserkers),
   (.coat, berserkers), (.gloves, berserkers),
   (.leggings, berserkers), (.boots, berserkers)]
```

This is trivially supported by the solver architecture: fixing a slot to a singleton domain before propagation means propagation will immediately exploit the fixed information. The rune propagator, for instance, will see that all armour slots have the same rune (whatever rune Berserker armour uses) and skip rune selection entirely.

The search space shrinks dramatically: if 6 of the 14 slots are fixed, the remaining space is $40^8 \approx 6.6 \times 10^{12}$ instead of $40^{14} \approx 6.9 \times 10^{22}$—and propagation typically prunes this much further.

Exercise

Implement what-if mode. Solve the following scenario: a Warrior with full Berserker armour (fixed) wants to optimise trinkets and weapons for Power $\geq$ 3000, Precision $\geq$ 2200, Ferocity $\geq$ 1500. How many B&B nodes does the solver explore compared to the unconstrained problem?

14.4 Pareto Frontier Mode

Use the scalarisation sweep from Chapter 9 with the full solver. For each weight vector, solve the scalarised problem to optimality. Output: the set of non-dominated builds across all weight vectors—an approximation of the Pareto frontier.

Lean 4 (Mathlib) --- Pareto Frontier Mode

```
/-- Generate the Pareto frontier by sweeping scalarisation weights,
    using the `wMissPerStat` field of GearProblem (Chapter 2). -/
def paretoFrontierMode (problem : GearProblem)
    (statA statB : StatType) (nPoints : Nat := 20)
    : List (Build × Nat × Nat) :=
  (List.range (nPoints + 1)).filterMap fun i =>
    let w := (i.toFloat) / nPoints.toFloat  -- weight for statA miss
    let weightedProblem := { problem with
      wMissPerStat := fun s =>
        -- `Float.toNat` does not exist in v4.31; go via UInt64.
        if s == statA then (w * 100).toUInt64.toNat
        else if s == statB then ((1.0 - w) * 100).toUInt64.toNat
        else 50 }  -- (the `}` used to sit INSIDE this comment!)
    match solve weightedProblem with
    | .optimal build pen =>
      some (build, statTotal build statA, statTotal build statB)
    | .infeasible => none

/-- Filter to non-dominated solutions. -/
def filterDominated (results : List (Build × Nat × Nat))
    : List (Build × Nat × Nat) :=
  results.filter fun (_, a1, b1) =>
    ¬ results.any fun (_, a2, b2) =>
      (a2 ≥ a1 ∧ b2 ≥ b1) ∧ (a2 > a1 ∨ b2 > b1)
```

Two fixes: `Float.toNat` does not exist in v4.31 (route through `.toUInt64.toNat`), and the earlier edition's closing brace `}` was *inside* the trailing comment, so the record update never closed. For `wMissPerStat` to actually steer the sweep, the scalarised `solve` and `totalPenalty` must read it in place of the global `wMiss`—a small extension left to the reader.

Each point on the frontier represents a genuinely different gear philosophy. On the Power/Vitality frontier, one end is full Berserker (maximum damage, minimum health); the other is full Soldier or Minstrel (maximum health, minimum damage). The interior points are the interesting

ones—mixed builds that trade damage for survivability at different rates.

Intuition

The Pareto frontier mode is expensive: it runs the full solver n times. But most of the work is shared: the propagation at the root node is identical for all weight vectors (it depends on stat targets, not weights), and the warm-start from SA can be reused across adjacent weight vectors.

14.5 Real-World Analogues

The solver architecture—propagation → B&B → LP bounding → heuristic warm-start—is not specific to GW2. It is the core design of industrial solvers like Gurobi, CPLEX, and Google OR-Tools. Problems with the same structure include:

- **Portfolio optimisation:** choose discrete asset lots to meet return targets at minimum risk.
- **Cloud provisioning:** select VM types to meet resource requirements at minimum cost.
- **Sports team composition:** select players within a salary cap to maximise expected performance.
- **Hardware configuration:** choose components (CPU, RAM, storage) to meet workload requirements.

In each case, the decision variables are discrete, constraints couple them, and the objective involves meeting targets with asymmetric penalties.

Intuition

You haven't just built a gear optimiser. You've built a miniature constraint programming / mixed-integer programming solver, verified in Lean 4. The techniques generalise to any problem with the same structure.

Appendix A

WvW Stat Combo Reference

The 40 WvW-legal stat combinations at Ascended rarity.

A.1 Triple-Attribute (1 Major, 2 Minor)

Name	Major	Minor 1	Minor 2
Berserker's	Power	Precision	Ferocity
Zealot's	Power	Precision	Healing Power
Soldier's	Power	Toughness	Vitality
Valkyrie	Power	Vitality	Ferocity
Harrier's	Power	Healing Power	Concentration
Captain's	Precision	Power	Toughness
Rampager's	Precision	Power	Condition Damage
Assassin's	Precision	Power	Ferocity
Knight's	Toughness	Power	Precision
Cavalier's	Toughness	Power	Ferocity
Nomad's	Toughness	Vitality	Healing Power
Settler's	Toughness	Condition Damage	Healing Power
Giver's	Toughness	Healing Power	Concentration
Sentinel's	Vitality	Power	Toughness
Shaman's	Vitality	Condition Damage	Healing Power
Sinister	Condition Damage	Power	Precision
Carrion	Condition Damage	Power	Vitality
Rabid	Condition Damage	Precision	Toughness
Dire	Condition Damage	Toughness	Vitality
Apostate's	Condition Damage	Toughness	Healing Power
Bringer's	Expertise	Precision	Vitality
Cleric's	Healing Power	Power	Toughness
Magi's	Healing Power	Precision	Vitality
Apothecary's	Healing Power	Toughness	Condition Damage

A.2 Quadruple-Attribute (2 Major, 2 Minor)

Name	Major 1	Major 2	Minor 1	Minor 2
Commander's	Power	Precision	Toughness	Concentration
Demolisher	Power	Precision	Toughness	Ferocity
Marauder	Power	Precision	Vitality	Ferocity

Name	Major 1	Major 2	Minor 1	Minor 2
Vigilant	Power	Toughness	Concentration	Expertise
Crusader	Power	Toughness	Ferocity	Healing Power
Wanderer's	Power	Vitality	Toughness	Concentration
Diviner's	Power	Concentration	Precision	Ferocity
Dragon's	Power	Ferocity	Precision	Vitality
Viper's	Power	Condition Damage	Precision	Expertise
Grieving	Power	Condition Damage	Precision	Ferocity
Marshal's	Power	Healing Power	Precision	Condition Damage
Seraph	Precision	Condition Damage	Concentration	Healing Power
Trailblazer's	Toughness	Condition Damage	Vitality	Expertise
Minstrel's	Toughness	Healing Power	Vitality	Concentration
Ritualist's	Vitality	Condition Damage	Concentration	Expertise
Plaguedoctor's	Vitality	Condition Damage	Healing Power	Concentration

A.3 Celestial (WvW Version)

All seven stats receive equal coefficients: Power, Precision, Ferocity, Vitality, Toughness, Healing Power, Condition Damage.

Note: The WvW version of Celestial excludes Expertise and Concentration, which are included in the PvE version.

Appendix B

Cross-Reference Map

Lattice Book Concept	**This Book Application**
Cell with join-semilattice values	Gear slot domains (`Finset StatCombo`, reverse inclusion)
Propagator (monotone function)	Rune, LinearGeq, LinearLeq, infusion-budget propagators
Knaster–Tarski fixed point	Propagation termination + correctness
Belnap's 4-valued lattice	3-valued SAT assignment lattice
Product lattice	Combined gear-slot domain lattice
Detecting $\top$ (contradiction)	Domain goes empty $\rightarrow$ prune this branch
Monotone convergence	Each B&B node runs propagation to fixpoint
`Cell.update` (join new info)	Propagating constraints narrows domains
Lattice height = termination bound	Total domain size bounds propagation steps
Set-of-possible-values lattice	Exactly the domain-reduction lattice for gear slots
Partial order	Pareto dominance, subproblem refinement, vertex ordering

Appendix C

A Lean 4 Primer

This appendix is a zero-prerequisites tour of every Lean 4 construct and tactic this book uses. Each entry says what it does, shows a small self-contained example (every example here is machine-checked—see the `Primer` namespace of `OptimizationVerification.lean`), and, where it illuminates, shows the lower-level term the tactic is sugar for. Tactics are not magic: they are programs that build proof terms.

C.1 Declarations

`inductive` — new types

An `inductive` declaration creates a type from a list of *constructors* (the only ways to build its values). Lean auto-generates an induction principle (the *recursor*).

Lean 4

```
inductive Colour where
  | red | green | blue
  deriving DecidableEq, Repr
```

`deriving Repr` synthesises a printer (so `#eval` can display values); `deriving DecidableEq` synthesises a decidable `=`.

`def` and pattern matching

A `def` introduces a function or value, usually by *pattern matching*: one equation per constructor. Recursion is allowed when an argument structurally shrinks; otherwise you must write `partial def` (as the solver's search loops do) and forfeit the ability to reason about the function in proofs.

Lean 4

```
def Colour.code : Colour → Nat
  | .red => 0 | .green => 1 | .blue => 2
```

The leading dot in `.red` means "the constructor of the type being matched".

`structure` and `instance`

A `structure` is a record; a *type class* is an interface and an `instance` registers an implementation. The anonymous constructor `⟨…⟩` builds a structure (and also proves `∧` and `∃`: `⟨w, h⟩` supplies a witness and a proof).

`theorem`, `example`, `Prop`

In Lean, propositions are types (of sort `Prop`) and proofs are terms: to prove P is to build a term of type P. A `theorem` is a named proof; an `example` is anonymous. A proof is either a direct term or a *tactic block* introduced by `by`.

Lean 4

```
example (p q : Prop) (hp : p) (h : p → q) : q := h hp    -- term style
example (p q : Prop) : p → (p → q) → q := by             -- tactic style
  intro hp h
  exact h hp
```

C.2 Core Tactics

`intro` — peel off hypotheses

If the goal is `p → q` (or `∀ x, P x`), `intro hp` moves the antecedent into context, leaving goal `q`. It is the tactic form of `fun hp => …`.

`exact` — supply the proof term

`exact h` closes the goal with `h`, which must match exactly.

`constructor` — build a structure/conjunction goal

When the goal is a structure or `∧`, `constructor` splits it into one subgoal per field. The two bullets `·` below discharge each conjunct.

Lean 4

```
example (p q : Prop) (hp : p) (hq : q) : p ∧ q := by
  constructor
  · exact hp
  · exact hq
```

`obtain` — destructure a hypothesis

`obtain ⟨n, hn⟩ := h` takes apart an `∃` or `∧`. It desugars to `Exists.elim` (or a `match`):

Lean 4

```
example (P : Nat → Prop) (h : ∃ n, P n) : ∃ m, P m := by
  obtain ⟨n, hn⟩ := h
  exact ⟨n, hn⟩
-- desugaring:
example (P : Nat → Prop) (h : ∃ n, P n) : ∃ m, P m :=
  Exists.elim h (fun n hn => ⟨n, hn⟩)
```

`unfold` — expand a definition

`unfold f` replaces `f` by its definition in the goal (then usually followed by `rfl` or `simp`).

Lean 4

```
def sq (n : Nat) : Nat := n * n
example : sq 3 = 9 := by unfold sq; rfl
```

`rfl` — true by computation

Proves `a = a` after both sides compute to the same normal form; definitional unfolding is free, so `2 + 3 = 5` is `rfl`.

`simp` — the simplifier

Rewrites with every `@[simp]` rule (plus any you pass in brackets) until nothing changes, then tries to close the goal. `simpa using h` simplifies both the goal and `h` and closes with it (used in the `Decidable` instance for dominance).

Lean 4

```
example (n : Nat) : n + 0 = n := by simp
```

`omega` — linear arithmetic over `Nat`/`Int`

A decision procedure for linear integer/natural arithmetic (with $+$, $-$, $\leq$, $<$, and multiplication by literals). It closes goals like the ones in this book's Pareto and LP-bound sanity checks.

Lean 4

```
example (a b : Nat) (h : a + 1 < b) : a < b := by omega
```

`split` — case on an `if`/`match` in the goal

When the goal contains `if c then … else …` (or a `match`), `split` creates one subgoal per branch, adding the branch condition as a hypothesis.

Lean 4

```
def clip (n : Nat) : Nat := if n < 10 then n else 10
example (n : Nat) : clip n ≤ 10 := by
  unfold clip
  split
  · omega   -- case n < 10
  · omega   -- case ¬ (n < 10)
```

`induction` / `cases` — and their recursor desugaring

`induction n with` splits into one case per constructor, giving the step case an induction hypothesis. Under the hood there is no primitive "induction": every `inductive` type `T` comes with a *recursor* `T.rec` (a function stating the induction principle), and the tactic merely applies it. `cases` is the same without the induction hypothesis.

Lean 4

```
theorem zero_add_nat (n : Nat) : 0 + n = n := by
  induction n with
  | zero => rfl
  | succ n ih => rw [Nat.add_succ, ih]

-- ...and its desugaring: applying the recursor directly.
theorem zero_add_nat' (n : Nat) : 0 + n = n :=
  Nat.rec (motive := fun n => 0 + n = n)
    rfl                                          -- base case
    (fun n ih => by                              -- step case
      show 0 + (n + 1) = n + 1
      rw [Nat.add_succ, ih])
    n
```

C.3 `sorry` — "unproven"

Warning

`sorry` closes *any* goal, unconditionally, by *admitting* it without proof. It is a placeholder, not a proof: Lean accepts the file but emits the warning `declaration uses 'sorry'`, and anything built on a `sorry` is logically unfounded. In this book, a `sorry` always marks a **specification**—a precisely stated theorem whose proof is left as an extended exercise. We never present a `sorry`-backed theorem as if it were machine-checked. Whenever you see `sorry`, read it as "this is asserted, not established".

Lean 4

```
-- Compiles, but emits `declaration uses 'sorry'`. NOT a real proof.
theorem unproven (n : Nat) : n + 0 = n := by sorry
```

C.4 Library Notes

`Finset` (finite sets), `SemilatticeSup`/`⊔` (join-semilattices), and `BinaryHeap` (a binary heap, from Batteries) are **not** core Lean—they require `import Mathlib` (Batteries ships with it). See Section 1.3 for project setup. `decidable_of_iff`, `List.all_eq_true`, and `List.any_eq_true` (used to make dominance decidable) are core Lean.